

Module - 3 Classification

MOHAMED YOUSUFF (AP/CI/SCOPE, VIT)

Contents

1	Feature Selection for Text Classification	1
1.1	Concept and Methods	1
1.2	Worked Example: Customer-Support Ticket Classification	2
1.3	Feature Scoring and Selection	3
2	Probabilistic and Naive Bayes Classifiers	11
2.1	Probabilistic View of Text Classification	11
2.2	Worked Example: Customer-Support Ticket Classification	12
2.3	Naive Bayes Computation	14
2.4	Classification of Query Data Points	20
3	Proximity-Based Classifiers	24
3.1	Core Principle	24
3.2	Worked Example and Vector Representation	25
3.3	k-Nearest Neighbor Classifier	27
3.4	Nearest Centroid Classifier	35
4	Classification of Linked and Web Data	40
4.1	Core Principle	40
4.2	Important Sources of Evidence	41
4.3	Worked Example: E-Commerce Web Page Classification	41
4.4	Content-Based Classification Score	43
4.5	Link-Based Classification Score	47
4.6	Combined Content-Link Classification	48
4.7	Collective Classification View	50
5	Meta-Algorithms for Text Classification	50
5.1	Types and Working Principles of Meta-Algorithms	51
5.2	Random Forest Classifier	58
5.3	AdaBoost Classifier	66
6	Content-Based Spam Email Classification Using Machine-Learning Algorithms	74

6.1	Core Principle	74
6.2	Dataset and Vector Representation	75
6.3	Logistic Regression Classifier	77
6.4	Support Vector Machine Classifier	81

1 Feature Selection for Text Classification

1.1 Concept and Methods

In text classification, each document is represented using terms obtained after preprocessing. Every unique term in the corpus may become a feature. However, all terms do not contribute equally to classification. Some terms strongly indicate a particular class, some terms appear in both classes, and some terms occur too rarely to provide strong statistical evidence.

Feature selection is the process of identifying discriminative terms from the vocabulary. A discriminative term is a term whose presence or absence helps separate one class from another. Feature selection does not create new features; it selects useful features from the existing vocabulary.

The standard process is:

Raw text dataset → Preprocessing → Vocabulary construction
→ Document-term representation → Feature scoring → Selected feature set

The vocabulary obtained after preprocessing is treated as the initial feature set. Feature selection methods are applied to all vocabulary terms.

Document frequency selects features based on the number of documents in which each term appears. It is simple, but it does not consider class labels.

Chi-square measures the association between a term and a class label. It gives high scores to terms that occur mostly in one class and rarely in another class.

Information gain measures how much a term reduces uncertainty about the class label. A term receives a high score if the presence or absence of that term makes the class more predictable.

Mutual information measures how much information is shared between a term and a class label. It evaluates the dependency between term occurrence and class membership.

Wrapper methods evaluate feature subsets by repeatedly training and testing a classifier. They are computationally expensive for text data because the number of possible feature subsets is very large.

Embedded methods perform feature selection during classifier training. Decision trees, random forests, and regularized linear models are common examples.

1.2 Worked Example: Customer-Support Ticket Classification

Consider a customer-support ticket classification problem. Each ticket belongs to one of two classes:

Refund, Delivery

The task is to identify useful terms for classifying future customer-support tickets.

Document	Raw Customer-Support Ticket	Class
D1	Refund is pending after payment cancellation.	Refund
D2	Money deducted from account but refund pending.	Refund
D3	Refund requested for damaged product payment.	Refund
D4	Delivery delay reported by courier tracking.	Delivery
D5	Package delivery tracking shows delay.	Delivery
D6	Courier shipment failed due to delivery delay.	Delivery

Table 1: Customer-support ticket dataset

Preprocessing

After lowercase conversion, punctuation removal, stop-word removal, and simple normalization such as converting *requested* to *request*, the documents become:

Document	Preprocessed Ticket	Class
D1	refund pending payment cancellation	Refund
D2	money deducted account refund pending	Refund
D3	refund request damaged product payment	Refund
D4	delivery delay reported courier tracking	Delivery
D5	package delivery tracking delay	Delivery
D6	courier shipment failed delivery delay	Delivery

Table 2: Preprocessed customer-support tickets

Vocabulary Construction

The vocabulary is constructed from all preprocessed tickets.

$V = [\text{refund, pending, payment, cancellation, money,}$
 $\text{deducted, account, request, damaged, product,}$
 $\text{delivery, delay, reported, courier, tracking,}$
 $\text{package, shipment, failed}]$

The vocabulary size is:

$$|V| = 18$$

All vocabulary terms are considered as the initial feature set.

Binary Term-Occurrence Matrix

The value 1 indicates that the term appears in the document, and 0 indicates that the term does not appear.

Doc	refund	pending	payment	cancellation	money	deducted	account	request	damaged	product	delivery	delay	reported	courier	tracking	package	shipment	failed	Class
D1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Refund
D2	1	1	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	Refund
D3	1	0	1	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	Refund
D4	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	Delivery
D5	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	1	0	0	Delivery
D6	0	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	1	1	Delivery

Table 3: Binary term-occurrence matrix

The dataset contains:

$$N = 6, \quad \text{Refund documents} = 3, \quad \text{Delivery documents} = 3$$

1.3 Feature Scoring and Selection

Document Frequency Method

Document frequency measures how many documents contain a term.

$$DF(t) = \text{Number of documents containing term } t$$

If the threshold is:

Term	Documents Containing the Term	DF
refund	D1, D2, D3	3
pending	D1, D2	2
payment	D1, D3	2
cancellation	D1	1
money	D2	1
deducted	D2	1
account	D2	1
request	D3	1
damaged	D3	1
product	D3	1
delivery	D4, D5, D6	3
delay	D4, D5, D6	3
reported	D4	1
courier	D4, D6	2
tracking	D4, D5	2
package	D5	1
shipment	D6	1
failed	D6	1

Table 4: Document frequency values for all vocabulary terms

$$DF(t) \geq 2$$

then the features selected by document frequency are:

refund, pending, payment, delivery, delay, courier, tracking

Document frequency removes many rare terms, but it does not consider class labels. Therefore, a term may be frequent but not necessarily useful for classification. Class-aware methods such as chi-square and information gain provide stronger evidence for text classification.

Chi-Square Method

The chi-square method measures the association between a term and a class label. For a term t and target class c , the chi-square score is:

$$\chi^2(t, c) = \frac{N(AD - BC)^2}{(A + C)(B + D)(A + B)(C + D)}$$

The target class is taken as:

$$c = \text{Refund}$$

The contingency table is:

	Term Present	Term Absent
Refund	A	C
Delivery	B	D

The values are interpreted as:

A = Refund documents containing the term

B = Delivery documents containing the term

C = Refund documents not containing the term

D = Delivery documents not containing the term

Detailed Chi-Square Computation for Four Terms*Term: refund*

The term *refund* appears in D1, D2, and D3.
All belong to Refund.

	refund Present	refund Absent
Refund	3	0
Delivery	0	3

$$A = 3, \quad B = 0, \quad C = 0, \quad D = 3$$

$$\begin{aligned}\chi^2 &= \frac{6((3)(3) - (0)(0))^2}{(3+0)(0+3)(3+0)(0+3)} \\ &= \frac{6(9)^2}{(3)(3)(3)(3)} = \frac{6(81)}{81} = 6.000\end{aligned}$$

Term: pending

The term *pending* appears in D1 and D2. Both
belong to Refund.

	pending Present	pending Absent
Refund	2	1
Delivery	0	3

$$A = 2, \quad B = 0, \quad C = 1, \quad D = 3$$

$$\begin{aligned}\chi^2 &= \frac{6((2)(3) - (0)(1))^2}{(2+1)(0+3)(2+0)(1+3)} \\ &= \frac{6(6)^2}{(3)(3)(2)(4)} = \frac{216}{72} = 3.000\end{aligned}$$

Term: delivery

The term *delivery* appears in D4, D5, and D6.
All belong to Delivery.

	delivery Present	delivery Absent
Refund	0	3
Delivery	3	0

$$A = 0, \quad B = 3, \quad C = 3, \quad D = 0$$

$$\begin{aligned}\chi^2 &= \frac{6((0)(0) - (3)(3))^2}{(0+3)(3+0)(0+3)(3+0)} \\ &= \frac{6(-9)^2}{(3)(3)(3)(3)} = \frac{6(81)}{81} = 6.000\end{aligned}$$

Term: failed

The term *failed* appears only in D6, which be-
longs to Delivery.

	failed Present	failed Absent
Refund	0	3
Delivery	1	2

$$A = 0, \quad B = 1, \quad C = 3, \quad D = 2$$

$$\begin{aligned}\chi^2 &= \frac{6((0)(2) - (1)(3))^2}{(0+3)(1+2)(0+1)(3+2)} \\ &= \frac{6(-3)^2}{(3)(3)(1)(5)} = \frac{54}{45} = 1.200\end{aligned}$$

Compact Chi-Square Computation for Remaining Terms

If the threshold is:

Term	A	B	C	D	Chi-Square Substitution	Score
payment	2	0	1	3	$\frac{6((2)(3) - (0)(1))^2}{(2+1)(0+3)(2+0)(1+3)} = \frac{216}{72}$	3.000
cancellation	1	0	2	3	$\frac{6((1)(3) - (0)(2))^2}{(1+2)(0+3)(1+0)(2+3)} = \frac{54}{45}$	1.200
money	1	0	2	3	$\frac{6((1)(3) - (0)(2))^2}{(1+2)(0+3)(1+0)(2+3)} = \frac{54}{45}$	1.200
deducted	1	0	2	3	$\frac{6((1)(3) - (0)(2))^2}{(1+2)(0+3)(1+0)(2+3)} = \frac{54}{45}$	1.200
account	1	0	2	3	$\frac{6((1)(3) - (0)(2))^2}{(1+2)(0+3)(1+0)(2+3)} = \frac{54}{45}$	1.200
request	1	0	2	3	$\frac{6((1)(3) - (0)(2))^2}{(1+2)(0+3)(1+0)(2+3)} = \frac{54}{45}$	1.200
damaged	1	0	2	3	$\frac{6((1)(3) - (0)(2))^2}{(1+2)(0+3)(1+0)(2+3)} = \frac{54}{45}$	1.200
product	1	0	2	3	$\frac{6((1)(3) - (0)(2))^2}{(1+2)(0+3)(1+0)(2+3)} = \frac{54}{45}$	1.200
delay	0	3	3	0	$\frac{6((0)(0) - (3)(3))^2}{(0+3)(3+0)(0+3)(3+0)} = \frac{486}{81}$	6.000
reported	0	1	3	2	$\frac{6((0)(2) - (1)(3))^2}{(0+3)(1+2)(0+1)(3+2)} = \frac{54}{45}$	1.200
courier	0	2	3	1	$\frac{6((0)(1) - (2)(3))^2}{(0+3)(2+1)(0+2)(3+1)} = \frac{216}{72}$	3.000
tracking	0	2	3	1	$\frac{6((0)(1) - (2)(3))^2}{(0+3)(2+1)(0+2)(3+1)} = \frac{216}{72}$	3.000
package	0	1	3	2	$\frac{6((0)(2) - (1)(3))^2}{(0+3)(1+2)(0+1)(3+2)} = \frac{54}{45}$	1.200
shipment	0	1	3	2	$\frac{6((0)(2) - (1)(3))^2}{(0+3)(1+2)(0+1)(3+2)} = \frac{54}{45}$	1.200

Table 5: Compact chi-square computation for remaining vocabulary terms

$$\chi^2(t) \geq 3.000$$

then the features selected by chi-square are:

refund, pending, payment, delivery, delay, courier, tracking

Information Gain Method

Information gain measures how much a term reduces uncertainty about the class label.

$$IG(t) = H(C) - H(C | t)$$

The entropy of the class variable is:

$$H(C) = - \sum_i P(c_i) \log_2 P(c_i)$$

Since there are three Refund and three Delivery documents:

$$P(\text{Refund}) = \frac{3}{6} = 0.5, \quad P(\text{Delivery}) = \frac{3}{6} = 0.5$$

$$H(C) = -[0.5 \log_2(0.5) + 0.5 \log_2(0.5)]$$

$$H(C) = -[0.5(-1) + 0.5(-1)]$$

$$H(C) = 1$$

The conditional entropy is:

$$H(C | t) = P(t = 1)H(C | t = 1) + P(t = 0)H(C | t = 0)$$

Detailed Information Gain Computation for Four Terms*Term: refund*When *refund* is present:

$$Refund = 3, \quad Delivery = 0$$

$$H(C \mid refund = 1) = 0$$

When *refund* is absent:

$$Refund = 0, \quad Delivery = 3$$

$$H(C \mid refund = 0) = 0$$

$$H(C \mid refund) = \frac{3}{6}(0) + \frac{3}{6}(0) = 0$$

$$IG(refund) = 1 - 0 = 1.000$$

*Term: delivery*When *delivery* is present:

$$Refund = 0, \quad Delivery = 3$$

$$H(C \mid delivery = 1) = 0$$

When *delivery* is absent:

$$Refund = 3, \quad Delivery = 0$$

$$H(C \mid delivery = 0) = 0$$

$$H(C \mid delivery) = \frac{3}{6}(0) + \frac{3}{6}(0) = 0$$

$$IG(delivery) = 1 - 0 = 1.000$$

Term: *pending*

When *pending* is present:

$$Refund = 2, \quad Delivery = 0$$

$$H(C \mid pending = 1) = 0$$

When *pending* is absent:

$$Refund = 1, \quad Delivery = 3$$

$$H(C \mid pending = 0) = 0.811$$

$$H(C \mid pending) = \frac{2}{6}(0) + \frac{4}{6}(0.811) = 0.541$$

$$IG(pending) = 1 - 0.541 = 0.459$$

Term: *failed*

When *failed* is present:

$$Refund = 0, \quad Delivery = 1$$

$$H(C \mid failed = 1) = 0$$

When *failed* is absent:

$$Refund = 3, \quad Delivery = 2$$

$$H(C \mid failed = 0) = 0.971$$

$$H(C \mid failed) = \frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809$$

$$IG(failed) = 1 - 0.809 = 0.191$$

Compact Information Gain Computation for Remaining Terms

Useful entropy values:

$$H(1,0) = H(0,1) = 0$$

$$H\left(\frac{1}{4}, \frac{3}{4}\right) = 0.811$$

$$H\left(\frac{2}{5}, \frac{3}{5}\right) = 0.971$$

If the threshold is:

$$IG(t) \geq 0.459$$

then the features selected by information gain are:

Term	Distribution When Present	Distribution When Absent	Conditional Entropy Computation	IG
payment	$R = 2, D = 0; H = 0$	$R = 1, D = 3; H = 0.811$	$\frac{2}{6}(0) + \frac{4}{6}(0.811) = 0.541; 1 - 0.541$	0.459
cancellation	$R = 1, D = 0; H = 0$	$R = 2, D = 3; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191
money	$R = 1, D = 0; H = 0$	$R = 2, D = 3; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191
deducted	$R = 1, D = 0; H = 0$	$R = 2, D = 3; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191
account	$R = 1, D = 0; H = 0$	$R = 2, D = 3; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191
request	$R = 1, D = 0; H = 0$	$R = 2, D = 3; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191
damaged	$R = 1, D = 0; H = 0$	$R = 2, D = 3; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191
product	$R = 1, D = 0; H = 0$	$R = 2, D = 3; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191
delay	$R = 0, D = 3; H = 0$	$R = 3, D = 0; H = 0$	$\frac{3}{6}(0) + \frac{3}{6}(0) = 0; 1 - 0$	1.000
reported	$R = 0, D = 1; H = 0$	$R = 3, D = 2; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191
courier	$R = 0, D = 2; H = 0$	$R = 3, D = 1; H = 0.811$	$\frac{3}{6}(0) + \frac{3}{6}(0.811) = 0.541; 1 - 0.541$	0.459
tracking	$R = 0, D = 2; H = 0$	$R = 3, D = 1; H = 0.811$	$\frac{3}{6}(0) + \frac{3}{6}(0.811) = 0.541; 1 - 0.541$	0.459
package	$R = 0, D = 1; H = 0$	$R = 3, D = 2; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191
shipment	$R = 0, D = 1; H = 0$	$R = 3, D = 2; H = 0.971$	$\frac{1}{6}(0) + \frac{5}{6}(0.971) = 0.809; 1 - 0.809$	0.191

Table 6: Compact information gain computation for remaining vocabulary terms

refund, pending, payment, delivery, delay, courier, tracking

2 Probabilistic and Naive Bayes Classifiers

2.1 Probabilistic View of Text Classification

A *probabilistic classifier* assigns a document to the class that has the highest probability after observing the words in the document. In text classification, a document is represented using terms obtained after preprocessing. The classifier learns the relationship between words and class labels from previously labelled documents.

For a document d and class c , the classification decision is written as:

$$Class(d) = \arg \max_c P(c | d)$$

where $P(c | d)$ represents the probability of class c after observing document d .

The *Naive Bayes classifier* is a probabilistic classifier based on Bayes' theorem. It assumes that the words in a document are conditionally independent given the class. Although this independence assumption is simplified, Naive Bayes works effectively for many text classification problems because text data usually contains many informative words.

Bayes' theorem is:

$$P(c | d) = \frac{P(d | c) \cdot P(c)}{P(d)}$$

where:

$P(c | d)$ = probability of class c given document d

$P(d | c)$ = probability of document d given class c

$P(c)$ = prior probability of class c

$P(d)$ = probability of document d

For classification, $P(d)$ is common for all classes. Therefore, the comparison is based on:

$$P(c) \cdot P(d | c)$$

If the document contains words $w_1, w_2, \dots, w_n$, the Naive Bayes assumption gives:

$$P(d | c) = P(w_1 | c) \cdot P(w_2 | c) \cdots P(w_n | c)$$

Therefore, the classification rule becomes:

$$\text{Class}(d) = \arg \max_c P(c) \cdot P(w_1 | c) \cdot P(w_2 | c) \cdots P(w_n | c)$$

2.2 Worked Example: Customer-Support Ticket Classification

Consider a customer-support ticket classification problem. Each ticket belongs to one of two classes:

Refund, Delivery

The task is to classify a new ticket using the Naive Bayes classifier.

Document	Raw Customer-Support Ticket	Class
D1	Refund is pending after payment cancellation.	Refund
D2	Money deducted from account but refund pending.	Refund
D3	Refund requested for damaged product payment.	Refund
D4	Delivery delay reported by courier tracking.	Delivery
D5	Package delivery tracking shows delay.	Delivery
D6	Courier shipment failed due to delivery delay.	Delivery

Table 7: Customer-support ticket dataset

Preprocessing

After lowercase conversion, punctuation removal, stop-word removal, and simple normalization such as converting *requested* to *request*, the documents become:

Document	Preprocessed Ticket	Class
D1	refund pending payment cancellation	Refund
D2	money deducted account refund pending	Refund
D3	refund request damaged product payment	Refund
D4	delivery delay reported courier tracking	Delivery
D5	package delivery tracking delay	Delivery
D6	courier shipment failed delivery delay	Delivery

Table 8: Preprocessed customer-support tickets

Vocabulary Construction

The vocabulary is constructed from all preprocessed tickets.

$V = [\text{refund, pending, payment, cancellation, money,}$
 $\text{deducted, account, request, damaged, product,}$
 $\text{delivery, delay, reported, courier, tracking,}$
 $\text{package, shipment, failed}]$

The vocabulary size is:

$$|V| = 18$$

All vocabulary terms are considered in the Naive Bayes computation. No shortened or manually selected vocabulary is used.

2.3 Naive Bayes Computation

Class Prior Probability

There are six training documents.

$$N = 6$$

Number of Refund documents:

$$N_{\text{Refund}} = 3$$

Number of Delivery documents:

$$N_{\text{Delivery}} = 3$$

Therefore:

$$P(\text{Refund}) = \frac{3}{6} = 0.5$$

$$P(\text{Delivery}) = \frac{3}{6} = 0.5$$

Both classes have equal prior probability.

Word Counts in Each Class

The Refund class contains D1, D2, and D3.

$D1$ = refund pending payment cancellation

$D2$ = money deducted account refund pending

$D3$ = refund request damaged product payment

The Delivery class contains D4, D5, and D6.

$D4$ = delivery delay reported courier tracking

$D5$ = package delivery tracking delay

$D6$ = courier shipment failed delivery delay

The complete word-count table is shown below.

Term	Count in Refund	Count in Delivery
refund	3	0
pending	2	0
payment	2	0
cancellation	1	0
money	1	0
deducted	1	0
account	1	0
request	1	0
damaged	1	0
product	1	0
delivery	0	3
delay	0	3
reported	0	1
courier	0	2
tracking	0	2
package	0	1
shipment	0	1
failed	0	1

Table 9: Word counts in Refund and Delivery classes

Total word count in Refund class:

$$Total_{\text{Refund}} = 3 + 2 + 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1$$

$$Total_{\text{Refund}} = 14$$

Total word count in Delivery class:

$$Total_{\text{Delivery}} = 3 + 3 + 1 + 2 + 2 + 1 + 1 + 1$$

$$Total_{\text{Delivery}} = 14$$

Laplace Smoothing

Some words may not appear in a particular class. Without smoothing, the probability of such words becomes zero. If any word probability becomes zero, the complete document probability may become zero. To avoid this issue, *Laplace smoothing* is applied.

The smoothed probability is:

$$P(w | c) = \frac{\text{Count}(w, c) + 1}{\text{Total word count in class } c + |V|}$$

Here:

$$|V| = 18$$

For the Refund class:

$$Total_{\text{Refund}} + |V| = 14 + 18 = 32$$

For the Delivery class:

$$Total_{\text{Delivery}} + |V| = 14 + 18 = 32$$

Therefore, all conditional probabilities are computed using denominator 32.

Conditional Probability Computation with Laplace Smoothing

Some terms may not appear in a particular class. For example, the term *delivery* does not appear in the Refund class, and the term *refund* does not appear in the Delivery class. Without smoothing, such terms would get zero probability. If one term probability becomes zero, the complete Naive Bayes score becomes zero. To avoid this problem, *Laplace smoothing* is applied.

The smoothed conditional probability is:

$$P(w | c) = \frac{\text{Count}(w, c) + 1}{\text{Total word count in class } c + |V|}$$

Here, the vocabulary size is:

$$|V| = 18$$

The total word count in the Refund class is:

$$\text{Total}_{\text{Refund}} = 14$$

The total word count in the Delivery class is:

$$\text{Total}_{\text{Delivery}} = 14$$

Therefore, the denominator for both classes is:

$$14 + 18 = 32$$

Hence:

$$P(w | \text{Refund}) = \frac{\text{Count}(w, \text{Refund}) + 1}{32}$$

$$P(w | \text{Delivery}) = \frac{\text{Count}(w, \text{Delivery}) + 1}{32}$$

Detailed Conditional Probability Computations

For the term *refund*:

$$\text{Count}(\text{refund}, \text{Refund}) = 3$$

$$P(\text{refund} \mid \text{Refund}) = \frac{3 + 1}{32} = \frac{4}{32} = 0.125$$

$$\text{Count}(\text{refund}, \text{Delivery}) = 0$$

$$P(\text{refund} \mid \text{Delivery}) = \frac{0 + 1}{32} = \frac{1}{32} = 0.031$$

For the term *pending*:

$$\text{Count}(\text{pending}, \text{Refund}) = 2$$

$$P(\text{pending} \mid \text{Refund}) = \frac{2 + 1}{32} = \frac{3}{32} = 0.094$$

$$\text{Count}(\text{pending}, \text{Delivery}) = 0$$

$$P(\text{pending} \mid \text{Delivery}) = \frac{0 + 1}{32} = \frac{1}{32} = 0.031$$

For the term *delivery*:

$$\text{Count}(\text{delivery}, \text{Refund}) = 0$$

$$P(\text{delivery} \mid \text{Refund}) = \frac{0 + 1}{32} = \frac{1}{32} = 0.031$$

$$\text{Count}(\text{delivery}, \text{Delivery}) = 3$$

$$P(\text{delivery} \mid \text{Delivery}) = \frac{3 + 1}{32} = \frac{4}{32} = 0.125$$

For the term *courier*:

$$\text{Count}(\text{courier}, \text{Refund}) = 0$$

$$P(\text{courier} \mid \text{Refund}) = \frac{0 + 1}{32} = \frac{1}{32} = 0.031$$

$$\text{Count}(\text{courier}, \text{Delivery}) = 2$$

$$P(\text{courier} \mid \text{Delivery}) = \frac{2 + 1}{32} = \frac{3}{32} = 0.094$$

The same smoothing formula is applied to all vocabulary terms.

Complete Conditional Probability Table

Term	Count in Refund	$P(w \mid \text{Refund})$	Count in Delivery	$P(w \mid \text{Delivery})$
refund	3	$\frac{3+1}{32} = \frac{4}{32} = 0.125$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
pending	2	$\frac{2+1}{32} = \frac{3}{32} = 0.094$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
payment	2	$\frac{2+1}{32} = \frac{3}{32} = 0.094$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
cancellation	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
money	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
deducted	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
account	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
request	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
damaged	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
product	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$
delivery	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$	3	$\frac{3+1}{32} = \frac{4}{32} = 0.125$
delay	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$	3	$\frac{3+1}{32} = \frac{4}{32} = 0.125$
reported	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$
courier	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$	2	$\frac{2+1}{32} = \frac{3}{32} = 0.094$
tracking	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$	2	$\frac{2+1}{32} = \frac{3}{32} = 0.094$
package	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$
shipment	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$
failed	0	$\frac{0+1}{32} = \frac{1}{32} = 0.031$	1	$\frac{1+1}{32} = \frac{2}{32} = 0.063$

Table 10: Smoothed conditional probabilities using full vocabulary

The probabilities show that *refund*, *pending*, and *payment* are more likely under the Refund class, whereas *delivery*, *delay*, *courier*, and *tracking* are more likely under the Delivery class.

2.4 Classification of Query Data Points

Query 1: refund payment pending

Consider the query ticket:

refund payment pending

The query contains:

$$Q_1 = \{\text{refund, payment, pending}\}$$

The class priors are:

$$P(\text{Refund}) = 0.5$$

$$P(\text{Delivery}) = 0.5$$

The Refund score is:

$$\text{Score}(\text{Refund}) = P(\text{Refund}) \cdot P(\text{refund} \mid \text{Refund}) \cdot P(\text{payment} \mid \text{Refund}) \cdot P(\text{pending} \mid \text{Refund})$$

$$\text{Score}(\text{Refund}) = 0.5 \cdot \frac{4}{32} \cdot \frac{3}{32} \cdot \frac{3}{32}$$

$$\text{Score}(\text{Refund}) = 0.000549$$

The Delivery score is:

$$\text{Score}(\text{Delivery}) = P(\text{Delivery}) \cdot P(\text{refund} \mid \text{Delivery}) \cdot P(\text{payment} \mid \text{Delivery}) \cdot P(\text{pending} \mid \text{Delivery})$$

$$Score(\text{Delivery}) = 0.5 \cdot \frac{1}{32} \cdot \frac{1}{32} \cdot \frac{1}{32}$$

$$Score(\text{Delivery}) = 0.000015$$

Since:

$$0.000549 > 0.000015$$

the query ticket is classified as:

$$Q_1 \rightarrow \text{Refund}$$

The normalized class probabilities are:

$$P(\text{Refund} | Q_1) = \frac{0.000549}{0.000549 + 0.000015} = 0.973$$

$$P(\text{Delivery} | Q_1) = \frac{0.000015}{0.000549 + 0.000015} = 0.027$$

Therefore:

$$Q_1 \rightarrow \text{Refund}$$

Query 2: delivery courier tracking delay

Consider the query ticket:

delivery courier tracking delay

The query contains:

$$Q_2 = \{\text{delivery, courier, tracking, delay}\}$$

The Refund score is:

$$Score(\text{Refund}) = P(\text{Refund}) \cdot P(\text{delivery} \mid \text{Refund}) \cdot P(\text{courier} \mid \text{Refund}) \cdot P(\text{tracking} \mid \text{Refund}) \cdot P(\text{delay} \mid \text{Refund})$$

$$Score(\text{Refund}) = 0.5 \cdot \frac{1}{32} \cdot \frac{1}{32} \cdot \frac{1}{32} \cdot \frac{1}{32}$$

$$Score(\text{Refund}) = 0.000000477$$

The Delivery score is:

$$Score(\text{Delivery}) = P(\text{Delivery}) \cdot P(\text{delivery} \mid \text{Delivery}) \cdot P(\text{courier} \mid \text{Delivery}) \cdot P(\text{tracking} \mid \text{Delivery}) \cdot P(\text{delay} \mid \text{Delivery})$$

$$Score(\text{Delivery}) = 0.5 \cdot \frac{4}{32} \cdot \frac{3}{32} \cdot \frac{3}{32} \cdot \frac{4}{32}$$

$$Score(\text{Delivery}) = 0.0000687$$

Since:

$$0.0000687 > 0.000000477$$

the query ticket is classified as:

$$Q_2 \rightarrow \text{Delivery}$$

The normalized class probabilities are:

$$P(\text{Refund} \mid Q_2) = \frac{0.000000477}{0.000000477 + 0.0000687} = 0.007$$

$$P(\text{Delivery} \mid Q_2) = \frac{0.0000687}{0.000000477 + 0.0000687} = 0.993$$

Therefore:

$$Q_2 \rightarrow \text{Delivery}$$

Query 3: payment delivery delay

Consider the mixed query ticket:

payment delivery delay

The query contains:

$$Q_3 = \{\text{payment, delivery, delay}\}$$

The Refund score is:

$$\text{Score}(\text{Refund}) = P(\text{Refund}) \cdot P(\text{payment} \mid \text{Refund}) \cdot P(\text{delivery} \mid \text{Refund}) \cdot P(\text{delay} \mid \text{Refund})$$

$$\text{Score}(\text{Refund}) = 0.5 \cdot \frac{3}{32} \cdot \frac{1}{32} \cdot \frac{1}{32}$$

$$\text{Score}(\text{Refund}) = 0.000046$$

The Delivery score is:

$$\text{Score}(\text{Delivery}) = P(\text{Delivery}) \cdot P(\text{payment} \mid \text{Delivery}) \cdot P(\text{delivery} \mid \text{Delivery}) \cdot P(\text{delay} \mid \text{Delivery})$$

$$\text{Score}(\text{Delivery}) = 0.5 \cdot \frac{1}{32} \cdot \frac{4}{32} \cdot \frac{4}{32}$$

$$\text{Score}(\text{Delivery}) = 0.000244$$

Since:

$$0.000244 > 0.000046$$

the query ticket is classified as:

$$Q_3 \rightarrow \text{Delivery}$$

The normalized class probabilities are:

$$P(\text{Refund} | Q_3) = \frac{0.000046}{0.000046 + 0.000244} = 0.158$$

$$P(\text{Delivery} | Q_3) = \frac{0.000244}{0.000046 + 0.000244} = 0.842$$

Although the query contains the word *payment*, the words *delivery* and *delay* provide stronger evidence for the Delivery class. Therefore, the final class is Delivery.

Interpretation

Naive Bayes classifies a text document by combining prior class probability with word likelihoods. In this example, the full vocabulary obtained after preprocessing is used for probability estimation. Laplace smoothing prevents zero probabilities for words that are absent in a class.

The classifier assigns Refund to a query when refund-related terms such as *refund*, *payment*, and *pending* provide stronger probability evidence. It assigns Delivery when delivery-related terms such as *delivery*, *delay*, *courier*, and *tracking* dominate the query. Mixed tickets are handled by comparing the total probabilistic evidence from both classes.

3 Proximity-Based Classifiers

3.1 Core Principle

Proximity-based classifiers classify a new document by comparing it with already labelled training documents. The term *proximity* means nearness. If a new document is closer to documents of a

particular class, it is assigned to that class.

In text classification, raw text is first converted into a numerical vector. After vectorization, similarity or distance is computed between documents. A document may be compared directly with training documents, as in *k-nearest neighbor classification*, or with class representatives, as in the *nearest centroid classifier*.

For text data, *cosine similarity* is commonly used because document-term vectors are usually high-dimensional and sparse.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{||A|| \cdot ||B||}$$

where:

$A \cdot B$ = dot product of document vectors

$||A||$ = magnitude of vector A

$||B||$ = magnitude of vector B

Cosine distance is:

$$d_{\text{cosine}}(A, B) = 1 - \text{Cosine Similarity}(A, B)$$

A larger cosine similarity indicates greater similarity. A smaller cosine distance indicates greater distance-based closeness.

3.2 Worked Example and Vector Representation

Consider a customer-support ticket classification problem. Each ticket belongs to one of two classes:

Refund, Delivery

Document	Raw Customer-Support Ticket	Class
D1	Refund is pending after payment cancellation.	Refund
D2	Money deducted from account but refund pending.	Refund
D3	Refund requested for damaged product payment.	Refund
D4	Delivery delay reported by courier tracking.	Delivery
D5	Package delivery tracking shows delay.	Delivery
D6	Courier shipment failed due to delivery delay.	Delivery

Table 11: Customer-support ticket dataset

Preprocessing

After lowercase conversion, punctuation removal, stop-word removal, and simple normalization such as converting *requested* to *request*, the documents become:

Document	Preprocessed Ticket	Class
D1	refund pending payment cancellation	Refund
D2	money deducted account refund pending	Refund
D3	refund request damaged product payment	Refund
D4	delivery delay reported courier tracking	Delivery
D5	package delivery tracking delay	Delivery
D6	courier shipment failed delivery delay	Delivery

Table 12: Preprocessed customer-support tickets

Vocabulary Construction

The vocabulary is constructed from all preprocessed documents.

$$V = [\text{refund, pending, payment, cancellation, money,} \\ \text{deducted, account, request, damaged, product,} \\ \text{delivery, delay, reported, courier, tracking,} \\ \text{package, shipment, failed}]$$

The vocabulary size is:

$$|V| = 18$$

Each ticket is represented using this full vocabulary. No shortened vocabulary is used.

Binary Document-Term Matrix

The value 1 indicates that the term appears in the document, and 0 indicates that the term does not appear.

Doc	refund	pending	payment	cancellation	money	deducted	account	request	damaged	product	delivery	delay	reported	courier	tracking	package	shipment	failed	Class
D1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Refund
D2	1	1	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	Refund
D3	1	0	1	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	Refund
D4	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	Delivery
D5	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	1	0	0	Delivery
D6	0	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	1	1	Delivery

Table 13: Binary document-term matrix

The magnitudes of the training document vectors are:

$$||D1|| = \sqrt{4} = 2$$

$$||D2|| = \sqrt{5} = 2.236$$

$$||D3|| = \sqrt{5} = 2.236$$

$$||D4|| = \sqrt{5} = 2.236$$

$$||D5|| = \sqrt{4} = 2$$

$$||D6|| = \sqrt{5} = 2.236$$

3.3 k-Nearest Neighbor Classifier

The *k*-nearest neighbor classifier stores all labelled training documents. When a new document is given, it is compared with every training document. The *k* nearest documents are selected, and the class is decided by majority voting.

The procedure is:

New document $\rightarrow$ Vector representation $\rightarrow$ Similarity computation $\rightarrow$ Nearest neighbors $\rightarrow$ Majority class

For this example:

$$k = 3$$

The three most similar training documents are used for classification.

Query 1: refund payment pending

Consider the query ticket:

refund payment pending

The query vector contains three terms:

$$Q_1 = \{\text{refund}, \text{payment}, \text{pending}\}$$

Therefore:

$$\|Q_1\| = \sqrt{3} = 1.732$$

The cosine similarity between Q_1 and each training document is computed using:

$$\text{Cosine}(Q_1, D_i) = \frac{Q_1 \cdot D_i}{\|Q_1\| \cdot \|D_i\|}$$

Detailed computations for the most relevant comparisons are shown first.

For D_1 :

$$D_1 = \{\text{refund}, \text{pending}, \text{payment}, \text{cancellation}\}$$

The common terms are:

{refund, pending, payment}

$$Q_1 \cdot D1 = 3$$

$$\text{Cosine}(Q_1, D1) = \frac{3}{1.732 \cdot 2}$$

$$= \frac{3}{3.464} = 0.866$$

For $D2$:

$D2 = \{\text{money, deducted, account, refund, pending}\}$

The common terms are:

{refund, pending}

$$Q_1 \cdot D2 = 2$$

$$\text{Cosine}(Q_1, D2) = \frac{2}{1.732 \cdot 2.236}$$

$$= \frac{2}{3.873} = 0.516$$

For $D4$:

$D4 = \{\text{delivery, delay, reported, courier, tracking}\}$

There is no common term between Q_1 and $D4$.

$$Q_1 \cdot D4 = 0$$

$$\text{Cosine}(Q_1, D4) = 0$$

The complete similarity table is:

Document	Common Terms with Q_1	Dot Product	Cosine Computation	Class
D1	refund, pending, payment	3	$\frac{3}{1.732 \cdot 2} = 0.866$	Refund
D2	refund, pending	2	$\frac{2}{1.732 \cdot 2.236} = 0.516$	Refund
D3	refund, payment	2	$\frac{2}{1.732 \cdot 2.236} = 0.516$	Refund
D4	none	0	0	Delivery
D5	none	0	0	Delivery
D6	none	0	0	Delivery

Table 14: Cosine similarity values for Q_1

The three nearest neighbors are:

$$D1, \quad D2, \quad D3$$

Their classes are:

$$\text{Refund}, \quad \text{Refund}, \quad \text{Refund}$$

Therefore:

$$Q_1 \rightarrow \text{Refund}$$

Query 2: delivery courier tracking delay

Consider the query ticket:

delivery courier tracking delay

The query vector contains four terms:

$$Q_2 = \{\text{delivery, courier, tracking, delay}\}$$

Therefore:

$$||Q_2|| = \sqrt{4} = 2$$

For $D4$:

$$D4 = \{\text{delivery, delay, reported, courier, tracking}\}$$

The common terms are:

$$\{\text{delivery, delay, courier, tracking}\}$$

$$Q_2 \cdot D4 = 4$$

$$\text{Cosine}(Q_2, D4) = \frac{4}{2 \cdot 2.236}$$

$$= \frac{4}{4.472} = 0.894$$

For $D5$:

$$D5 = \{\text{package, delivery, tracking, delay}\}$$

The common terms are:

$$\{\text{delivery, tracking, delay}\}$$

$$Q_2 \cdot D5 = 3$$

$$\text{Cosine}(Q_2, D5) = \frac{3}{2 \cdot 2}$$

$$= \frac{3}{4} = 0.750$$

For $D1$:

$$D1 = \{\text{refund, pending, payment, cancellation}\}$$

There is no common term between Q_2 and $D1$.

$$Q_2 \cdot D1 = 0$$

$$\text{Cosine}(Q_2, D1) = 0$$

The complete similarity table is:

Document	Common Terms with Q_2	Dot Product	Cosine Computation	Class
D1	none	0	0	Refund
D2	none	0	0	Refund
D3	none	0	0	Refund
D4	delivery, delay, courier, tracking	4	$\frac{4}{2 \cdot 2.236} = 0.894$	Delivery
D5	delivery, delay, tracking	3	$\frac{3}{2 \cdot 2} = 0.750$	Delivery
D6	delivery, delay, courier	3	$\frac{3}{2 \cdot 2.236} = 0.671$	Delivery

Table 15: Cosine similarity values for Q_2

The three nearest neighbors are:

$$D4, \quad D5, \quad D6$$

Their classes are:

$$\text{Delivery, Delivery, Delivery}$$

Therefore:

$$Q_2 \rightarrow \text{Delivery}$$

Query 3: payment delivery delay

Consider the mixed query ticket:

payment delivery delay

The query contains one refund-related term and two delivery-related terms.

$$Q_3 = \{\text{payment, delivery, delay}\}$$

$$||Q_3|| = \sqrt{3} = 1.732$$

For $D1$:

$$D1 = \{\text{refund, pending, payment, cancellation}\}$$

The common term is:

$$\{\text{payment}\}$$

$$Q_3 \cdot D1 = 1$$

$$\text{Cosine}(Q_3, D1) = \frac{1}{1.732 \cdot 2}$$

$$= \frac{1}{3.464} = 0.289$$

For $D5$:

$$D5 = \{\text{package, delivery, tracking, delay}\}$$

The common terms are:

$$\{\text{delivery, delay}\}$$

$$Q_3 \cdot D5 = 2$$

$$\text{Cosine}(Q_3, D5) = \frac{2}{1.732 \cdot 2}$$

$$= \frac{2}{3.464} = 0.577$$

The complete similarity table is:

Document	Common Terms with Q_3	Dot Product	Cosine Computation	Class
D1	payment	1	$\frac{1}{1.732 \cdot 2} = 0.289$	Refund
D2	none	0	0	Refund
D3	payment	1	$\frac{1}{1.732 \cdot 2.236} = 0.258$	Refund
D4	delivery, delay	2	$\frac{2}{1.732 \cdot 2.236} = 0.516$	Delivery
D5	delivery, delay	2	$\frac{2}{1.732 \cdot 2} = 0.577$	Delivery
D6	delivery, delay	2	$\frac{2}{1.732 \cdot 2.236} = 0.516$	Delivery

Table 16: Cosine similarity values for Q_3

The three nearest neighbors are:

$$D5, \quad D4, \quad D6$$

Their classes are:

$$\text{Delivery, Delivery, Delivery}$$

Therefore:

$$Q_3 \rightarrow \text{Delivery}$$

Although Q_3 contains the term *payment*, it is closer to Delivery documents because it also contains *delivery* and *delay*.

3.4 Nearest Centroid Classifier

The *nearest centroid classifier* represents each class using a centroid. A centroid is the average vector of all training documents in a class. A query document is assigned to the class whose centroid is nearest or most similar.

For a class C , the centroid is:

$$\text{Centroid}(C) = \frac{D_1 + D_2 + \dots + D_n}{n}$$

where $D_1, D_2, \dots, D_n$ are the document vectors belonging to that class.

In this example, cosine similarity is used to compare query documents with class centroids.

Refund Class Centroid

Refund documents are:

$$D_1, \quad D_2, \quad D_3$$

The Refund centroid is:

$$C_{\text{Refund}} = \frac{D_1 + D_2 + D_3}{3}$$

For example:

$$C_{\text{Refund}}(\text{refund}) = \frac{1 + 1 + 1}{3} = 1$$

$$C_{\text{Refund}}(\text{pending}) = \frac{1 + 1 + 0}{3} = 0.667$$

$$C_{\text{Refund}}(\text{payment}) = \frac{1 + 0 + 1}{3} = 0.667$$

$$C_{\text{Refund}}(\text{cancellation}) = \frac{1 + 0 + 0}{3} = 0.333$$

The full Refund centroid is shown below.

Centroid	refund	pending	payment	cancellation	money	deducted	account	request	damaged	product	delivery	delay	reported	courier	tracking	package	shipment	failed
C_{Refund}	1	0.667	0.667	0.333	0.333	0.333	0.333	0.333	0.333	0.333	0	0	0	0	0	0	0	0

Table 17: Refund class centroid

The squared magnitude of the Refund centroid is:

$$||C_{\text{Refund}}||^2 = 1^2 + 2(0.667)^2 + 7(0.333)^2$$

$$= 1 + 2(0.445) + 7(0.111)$$

$$= 1 + 0.890 + 0.777 = 2.667$$

$$||C_{\text{Refund}}|| = \sqrt{2.667} = 1.633$$

Delivery Class Centroid

Delivery documents are:

$$D4, \quad D5, \quad D6$$

The Delivery centroid is:

$$C_{\text{Delivery}} = \frac{D4 + D5 + D6}{3}$$

For example:

$$C_{\text{Delivery}}(\text{delivery}) = \frac{1 + 1 + 1}{3} = 1$$

$$C_{\text{Delivery}}(\text{delay}) = \frac{1 + 1 + 1}{3} = 1$$

$$C_{\text{Delivery}}(\text{courier}) = \frac{1 + 0 + 1}{3} = 0.667$$

$$C_{\text{Delivery}}(\text{tracking}) = \frac{1 + 1 + 0}{3} = 0.667$$

The full Delivery centroid is shown below.

Centroid	refund	pending	payment	cancellation	money	deducted	account	request	damaged	product	delivery	delay	reported	courier	tracking	package	shipment	failed
C_{Delivery}	0	0	0	0	0	0	0	0	0	0	1	1	0.333	0.667	0.667	0.333	0.333	0.333

Table 18: Delivery class centroid

The squared magnitude of the Delivery centroid is:

$$\|C_{\text{Delivery}}\|^2 = 2(1)^2 + 2(0.667)^2 + 4(0.333)^2$$

$$= 2 + 2(0.445) + 4(0.111)$$

$$= 2 + 0.890 + 0.444 = 3.334$$

$$\|C_{\text{Delivery}}\| = \sqrt{3.334} = 1.826$$

Centroid Classification of Query 1

$$Q_1 = \{\text{refund, payment, pending}\}$$

$$\|Q_1\| = \sqrt{3} = 1.732$$

Dot product with Refund centroid:

$$Q_1 \cdot C_{\text{Refund}} = 1 + 0.667 + 0.667 = 2.334$$

$$\text{Cosine}(Q_1, C_{\text{Refund}}) = \frac{2.334}{1.732 \cdot 1.633}$$

$$= \frac{2.334}{2.828} = 0.825$$

Dot product with Delivery centroid:

$$Q_1 \cdot C_{\text{Delivery}} = 0$$

$$\text{Cosine}(Q_1, C_{\text{Delivery}}) = 0$$

Since:

$$0.825 > 0$$

$$Q_1 \rightarrow \text{Refund}$$

Centroid Classification of Query 2

$$Q_2 = \{\text{delivery, courier, tracking, delay}\}$$

$$||Q_2|| = 2$$

Dot product with Refund centroid:

$$Q_2 \cdot C_{\text{Refund}} = 0$$

$$\text{Cosine}(Q_2, C_{\text{Refund}}) = 0$$

Dot product with Delivery centroid:

$$Q_2 \cdot C_{\text{Delivery}} = 1 + 0.667 + 0.667 + 1 = 3.334$$

$$\text{Cosine}(Q_2, C_{\text{Delivery}}) = \frac{3.334}{2 \cdot 1.826}$$

$$= \frac{3.334}{3.652} = 0.913$$

Since:

$$0.913 > 0$$

$$Q_2 \rightarrow \text{Delivery}$$

Centroid Classification of Query 3

$$Q_3 = \{\text{payment, delivery, delay}\}$$

$$||Q_3|| = \sqrt{3} = 1.732$$

Dot product with Refund centroid:

$$Q_3 \cdot C_{\text{Refund}} = 0.667$$

$$\text{Cosine}(Q_3, C_{\text{Refund}}) = \frac{0.667}{1.732 \cdot 1.633}$$

$$= \frac{0.667}{2.828} = 0.236$$

Dot product with Delivery centroid:

$$Q_3 \cdot C_{\text{Delivery}} = 1 + 1 = 2$$

$$\text{Cosine}(Q_3, C_{\text{Delivery}}) = \frac{2}{1.732 \cdot 1.826}$$

$$= \frac{2}{3.163} = 0.632$$

Since:

$$0.632 > 0.236$$

$$Q_3 \rightarrow \text{Delivery}$$

Interpretation

Proximity-based classifiers classify a document by measuring closeness to labelled examples or class representatives. The k-nearest neighbor classifier compares a query document with every training document and uses the majority class among the nearest documents. The nearest centroid classifier compares a query document with class centroids and selects the most similar class.

In the customer-support example, tickets containing *refund*, *pending*, and *payment* are close to Refund documents. Tickets containing *delivery*, *delay*, *courier*, and *tracking* are close to Delivery documents. For a mixed ticket such as *payment delivery delay*, the Delivery evidence is stronger because the query is more similar to Delivery documents and also closer to the Delivery centroid.

4 Classification of Linked and Web Data

4.1 Core Principle

Classification of linked and web data deals with documents that are connected to one another through hyperlinks, citations, references, tags, menus, or other web-based relationships. In ordinary text classification, a document is classified mainly by its own content. In linked and web data classification, the class of a page is decided using both the *content of the page* and the *links connected to the page*.

A web page is rarely isolated. It may contain textual content, anchor text, outgoing links, incoming links, metadata, and neighboring pages. These links provide additional classification evidence. For example, if a page contains the words *delivery*, *tracking*, and *support*, and it also links to several already known support pages, then the page is likely to belong to a support-related class.

The classification process can be represented as:

Web page → Content representation → Content-based score
 → Link representation → Link-based score → Final class

Thus, the final classification can be based on:

Textual evidence + Link evidence

4.2 Important Sources of Evidence

In linked and web data classification, the following sources are commonly used.

Page content refers to the visible text of the web page. It includes words and phrases in the page title, headings, paragraphs, product descriptions, support instructions, or article body.

Anchor text refers to the clickable text of a hyperlink. For example, an anchor text such as *track delivery* gives strong evidence that the linked page may be related to delivery support.

Outgoing links are links from the current page to other pages. If a page links mostly to support pages, it may also be support-related.

Incoming links are links from other pages to the current page. If many product pages link to a page, it may be product-related.

Neighbor labels are the class labels of connected pages. In many web classification problems, the labels of neighboring pages provide useful relational evidence.

4.3 Worked Example: E-Commerce Web Page Classification

Consider an e-commerce website containing two types of pages:

Support, Product

Support pages discuss refund, payment help, delivery tracking, return tickets, and courier support. Product pages discuss product price, discount, offers, mobile devices, laptops, and product launches.

A new page *W7* has to be classified.

Page	Raw Page Text	Class
W1	Refund payment support help page.	Support
W2	Delivery tracking courier support page.	Support
W3	Return refund ticket support page.	Support
W4	Mobile product price offer page.	Product
W5	Laptop product discount price page.	Product
W6	Smartphone product launch offer page.	Product

Table 19: Training web pages

W7 : Delivery tracking support for product.

The page also contains outgoing links to four already labelled pages:

$$W7 \rightarrow W1, \quad W7 \rightarrow W2, \quad W7 \rightarrow W3, \quad W7 \rightarrow W5$$

The known labels of these linked pages are:

$$W1 = \text{Support}, \quad W2 = \text{Support}, \quad W3 = \text{Support}, \quad W5 = \text{Product}$$

Preprocessing

After lowercase conversion, punctuation removal, and removal of general words such as *page* and *for*, the training pages become:

Page	Preprocessed Page Text	Class
W1	refund payment support help	Support
W2	delivery tracking courier support	Support
W3	return refund ticket support	Support
W4	mobile product price offer	Product
W5	laptop product discount price	Product
W6	smartphone product launch offer	Product

Table 20: Preprocessed training web pages

The query page becomes:

$$W7 = \{\text{delivery, tracking, support, product}\}$$

Vocabulary Construction

The vocabulary is constructed from all preprocessed training pages.

$V = [\text{refund, payment, support, help, delivery,}$
 $\text{tracking, courier, return, ticket, mobile,}$
 $\text{product, price, offer, laptop, discount,}$
 $\text{smartphone, launch}]$

The vocabulary size is:

$$|V| = 17$$

Binary Page-Term Matrix

Page	refund	payment	support	help	delivery	tracking	courier	return	ticket	mobile	product	price	offer	laptop	discount	smartphone	launch	Class
W1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	Support
W2	0	0	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	Support
W3	1	0	1	0	0	0	0	1	1	0	0	0	0	0	0	0	0	Support
W4	0	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	Product
W5	0	0	0	0	0	0	0	0	0	0	1	1	0	1	1	0	0	Product
W6	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	1	Product

Table 21: Binary page-term matrix

4.4 Content-Based Classification Score

A content-based classifier uses only the words present in the page. Here, a simple probabilistic content score is used to classify the new page $W7$. The same idea can be implemented using Naive Bayes, logistic regression, SVM, or other classifiers.

The class priors are:

$$P(\text{Support}) = \frac{3}{6} = 0.5$$

$$P(\text{Product}) = \frac{3}{6} = 0.5$$

The Support pages are W1, W2, and W3. The Product pages are W4, W5, and W6.

Total term count in Support pages:

$$Total_{\text{Support}} = 4 + 4 + 4 = 12$$

Total term count in Product pages:

$$Total_{\text{Product}} = 4 + 4 + 4 = 12$$

Using Laplace smoothing:

$$P(w | c) = \frac{\text{Count}(w, c) + 1}{\text{Total word count in class } c + |V|}$$

Since:

$$|V| = 17$$

the denominator for both classes is:

$$12 + 17 = 29$$

Detailed Conditional Probability Computation

For the term *delivery*:

$$\text{Count}(\text{delivery}, \text{Support}) = 1$$

$$P(\text{delivery} | \text{Support}) = \frac{1 + 1}{29} = \frac{2}{29} = 0.069$$

$$\text{Count}(\text{delivery}, \text{Product}) = 0$$

$$P(\text{delivery} | \text{Product}) = \frac{0 + 1}{29} = \frac{1}{29} = 0.034$$

For the term *support*:

$$\text{Count}(\text{support}, \text{Support}) = 3$$

$$P(\text{support} \mid \text{Support}) = \frac{3 + 1}{29} = \frac{4}{29} = 0.138$$

$$\text{Count}(\text{support}, \text{Product}) = 0$$

$$P(\text{support} \mid \text{Product}) = \frac{0 + 1}{29} = \frac{1}{29} = 0.034$$

For the term *product*:

$$\text{Count}(\text{product}, \text{Support}) = 0$$

$$P(\text{product} \mid \text{Support}) = \frac{0 + 1}{29} = \frac{1}{29} = 0.034$$

$$\text{Count}(\text{product}, \text{Product}) = 3$$

$$P(\text{product} \mid \text{Product}) = \frac{3 + 1}{29} = \frac{4}{29} = 0.138$$

Complete Conditional Probability Table

Content Score for Query Page W7

The query page is:

$$W7 = \{\text{delivery}, \text{tracking}, \text{support}, \text{product}\}$$

The Support content score is:

$$\begin{aligned} \text{Score}_{\text{content}}(\text{Support}) &= P(\text{Support}) \cdot P(\text{delivery} \mid \text{Support}) \cdot P(\text{tracking} \mid \text{Support}) \\ &\quad \cdot P(\text{support} \mid \text{Support}) \cdot P(\text{product} \mid \text{Support}) \end{aligned}$$

Term	Count in Support	$P(w \mid \text{Support})$	Count in Product	$P(w \mid \text{Product})$
refund	2	$\frac{2+1}{29} = \frac{3}{29} = 0.103$	0	$\frac{0+1}{29} = \frac{1}{29} = 0.034$
payment	1	$\frac{1+1}{29} = \frac{2}{29} = 0.069$	0	$\frac{1}{29} = 0.034$
support	3	$\frac{3+1}{29} = \frac{4}{29} = 0.138$	0	$\frac{1}{29} = 0.034$
help	1	$\frac{2}{29} = 0.069$	0	$\frac{1}{29} = 0.034$
delivery	1	$\frac{2}{29} = 0.069$	0	$\frac{1}{29} = 0.034$
tracking	1	$\frac{2}{29} = 0.069$	0	$\frac{1}{29} = 0.034$
courier	1	$\frac{2}{29} = 0.069$	0	$\frac{1}{29} = 0.034$
return	1	$\frac{2}{29} = 0.069$	0	$\frac{1}{29} = 0.034$
ticket	1	$\frac{2}{29} = 0.069$	0	$\frac{1}{29} = 0.034$
mobile	0	$\frac{1}{29} = 0.034$	1	$\frac{2}{29} = 0.069$
product	0	$\frac{1}{29} = 0.034$	3	$\frac{4}{29} = 0.138$
price	0	$\frac{1}{29} = 0.034$	2	$\frac{3}{29} = 0.103$
offer	0	$\frac{1}{29} = 0.034$	2	$\frac{3}{29} = 0.103$
laptop	0	$\frac{1}{29} = 0.034$	1	$\frac{2}{29} = 0.069$
discount	0	$\frac{1}{29} = 0.034$	1	$\frac{2}{29} = 0.069$
smartphone	0	$\frac{1}{29} = 0.034$	1	$\frac{2}{29} = 0.069$
launch	0	$\frac{1}{29} = 0.034$	1	$\frac{2}{29} = 0.069$

Table 22: Smoothed conditional probabilities for content-based scoring

$$Score_{\text{content}}(\text{Support}) = 0.5 \cdot \frac{2}{29} \cdot \frac{2}{29} \cdot \frac{4}{29} \cdot \frac{1}{29}$$

$$Score_{\text{content}}(\text{Support}) = \frac{8}{707281} = 0.00001131$$

The Product content score is:

$$Score_{\text{content}}(\text{Product}) = P(\text{Product}) \cdot P(\text{delivery} \mid \text{Product}) \cdot P(\text{tracking} \mid \text{Product}) \\ \cdot P(\text{support} \mid \text{Product}) \cdot P(\text{product} \mid \text{Product})$$

$$Score_{\text{content}}(\text{Product}) = 0.5 \cdot \frac{1}{29} \cdot \frac{1}{29} \cdot \frac{1}{29} \cdot \frac{4}{29}$$

$$Score_{\text{content}}(\text{Product}) = \frac{2}{707281} = 0.00000283$$

The normalized content-based probabilities are:

$$P_{\text{content}}(\text{Support} \mid W7) = \frac{0.00001131}{0.00001131 + 0.00000283} = 0.800$$

$$P_{\text{content}}(\text{Product} \mid W7) = \frac{0.00000283}{0.00001131 + 0.00000283} = 0.200$$

Thus, content evidence suggests:

$$W7 \rightarrow \text{Support}$$

4.5 Link-Based Classification Score

The query page $W7$ links to four labelled pages:

$$W7 \rightarrow W1, \quad W7 \rightarrow W2, \quad W7 \rightarrow W3, \quad W7 \rightarrow W5$$

The labels of linked pages are:

$$W1 = \text{Support}$$

$$W2 = \text{Support}$$

$$W3 = \text{Support}$$

$$W5 = \text{Product}$$

The link score of a class is calculated as:

$$\text{LinkScore}(c, W7) = \frac{\text{Number of linked pages belonging to class } c}{\text{Total number of linked pages}}$$

Total linked pages:

4

Support linked pages:

3

Product linked pages:

1

Therefore:

$$LinkScore(\text{Support}, W7) = \frac{3}{4} = 0.750$$

$$LinkScore(\text{Product}, W7) = \frac{1}{4} = 0.250$$

Thus, link evidence also suggests:

$$W7 \rightarrow \text{Support}$$

4.6 Combined Content-Link Classification

In linked web data classification, content evidence and link evidence can be combined using a weighted score.

$$FinalScore(c, W7) = \alpha \cdot ContentScore(c, W7) + (1 - \alpha) \cdot LinkScore(c, W7)$$

Here, normalized content scores are used:

$$ContentScore(\text{Support}, W7) = 0.800$$

$$ContentScore(\text{Product}, W7) = 0.200$$

For this example, let:

$$\alpha = 0.6$$

Therefore:

$$1 - \alpha = 0.4$$

This gives 60 percent importance to content evidence and 40 percent importance to link evidence.

Final Score for Support

$$FinalScore(\text{Support}, W7) = 0.6 \cdot 0.800 + 0.4 \cdot 0.750$$

$$= 0.480 + 0.300 = 0.780$$

Final Score for Product

$$FinalScore(\text{Product}, W7) = 0.6 \cdot 0.200 + 0.4 \cdot 0.250$$

$$= 0.120 + 0.100 = 0.220$$

Since:

$$0.780 > 0.220$$

the final classification is:

$$W7 \rightarrow \text{Support}$$

4.7 Collective Classification View

In linked web data, the class of one page may depend on the classes of connected pages. This idea is called *collective classification*. A page is not classified only from its own words; its neighbors also influence the classification.

The general idea is:

$$\text{Label}(W7) = f(\text{page content, linked pages, neighbor labels, anchor text})$$

For the query page $W7$, content already gives high support evidence. Link evidence strengthens this decision because three out of four linked pages are Support pages. Hence, both textual evidence and relational evidence point to the same final class.

Interpretation

Classification of linked and web data extends ordinary text classification by using both content and link structure. In the worked example, the new page $W7$ contains the terms *delivery*, *tracking*, and *support*, which are common in Support pages. It also links mostly to Support pages. The final score therefore classifies $W7$ as a Support page.

This type of classification is useful for web pages because links are meaningful signals. Page content explains what the page says, while link structure explains how the page is connected to other pages. A reliable web data classifier can combine both sources of evidence to make a stronger classification decision.

5 Meta-Algorithms for Text Classification

A *meta-algorithm* is a higher-level learning strategy that improves text classification by combining multiple classifiers, multiple training samples, or multiple decision rules. In ordinary classification, a single model is trained on the document-term matrix and used for prediction. In meta-algorithmic classification, several models or several versions of a model are created, and their outputs are combined to obtain a stronger final decision.

Text data is usually high-dimensional, sparse, and sometimes ambiguous. A document may contain words from more than one class. For example, in support-ticket classification, a ticket containing the terms *payment*, *delivery*, and *delay* may confuse a single classifier because *payment* may suggest

a refund issue, while *delivery* and *delay* may suggest a delivery issue. Meta-algorithms handle such situations by combining several models or several decision patterns rather than depending on one classifier alone.

5.1 Types and Working Principles of Meta-Algorithms

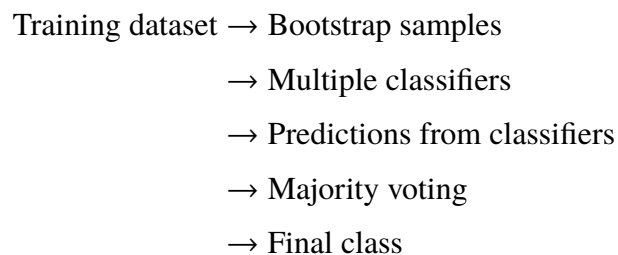
The major meta-algorithms used for text classification are *bagging*, *random forest*, *boosting*, *voting*, and *stacking*.

Bagging

Bagging, also called *Bootstrap Aggregating*, is a meta-algorithm that trains multiple classifiers using different samples of the same training dataset. Each sample is created by selecting documents from the original training set with replacement. Since sampling is done with replacement, some documents may appear more than once in a sample, while some documents may be left out.

Each bootstrap sample is used to train a separate base classifier. After training, all classifiers are used to classify a new document. The final class is usually decided by majority voting.

The working principle of bagging is:



Bagging is useful because it reduces the instability of a single classifier. If one classifier gives a wrong prediction due to a biased or noisy sample, other classifiers may correct it through voting. Bagging is especially effective when the base classifier is sensitive to changes in training data, such as a decision tree.

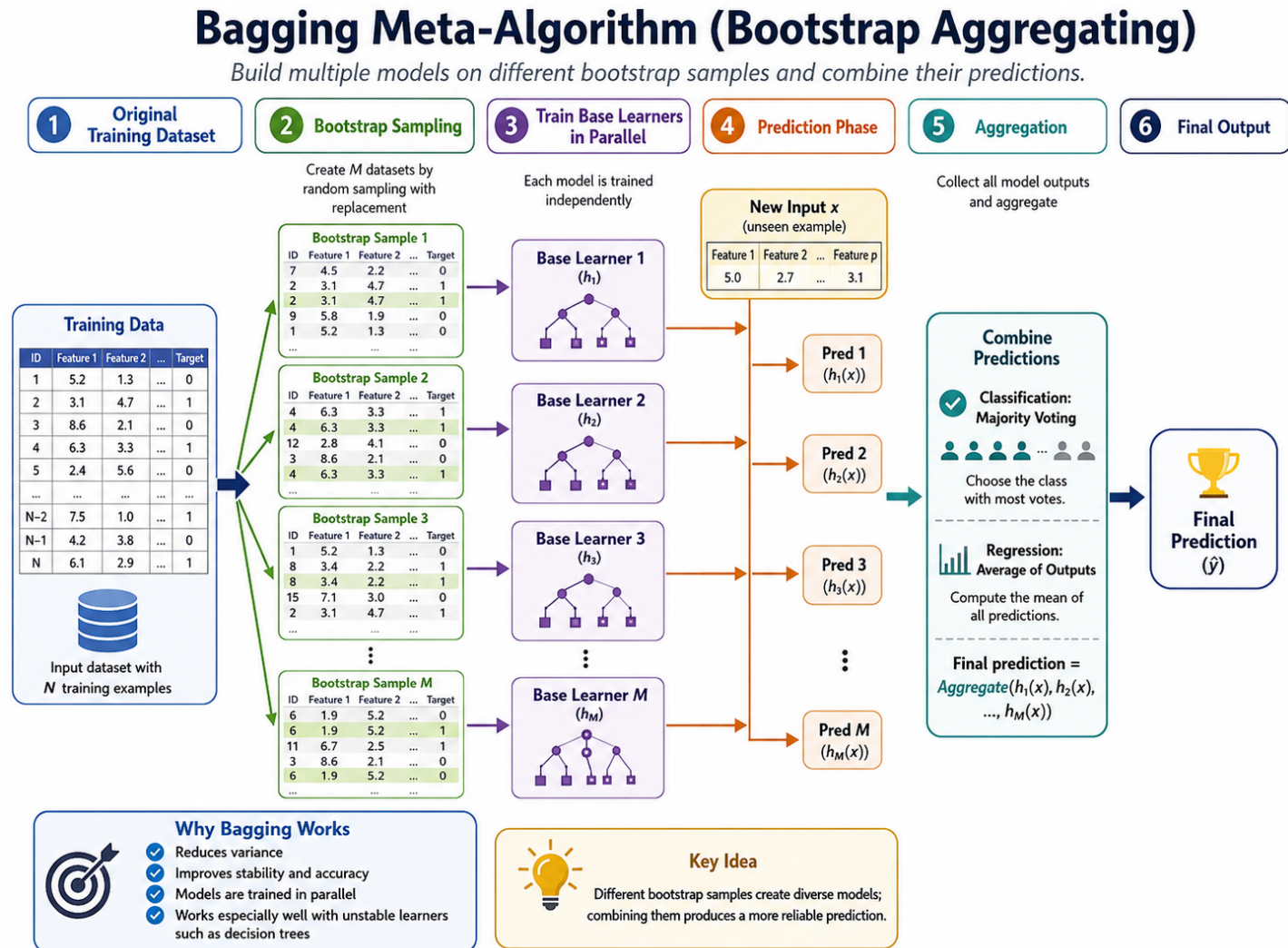


Figure 1: Working principle of bagging in text classification

Random Forest

Random Forest is an extension of bagging. It builds multiple decision trees using bootstrap samples. In addition to using different training samples, each tree is also allowed to consider only a random subset of features while making a split. This makes the trees more diverse.

In text classification, the features are usually words or terms. If every tree uses all terms, many trees may become similar. Random Forest avoids this by allowing each tree to examine only a subset of terms at each split. For example, one tree may focus on terms such as *refund*, *payment*, and *pending*, while another tree may focus on *delivery*, *delay*, and *courier*.

The working principle of Random Forest is:

Document-term matrix → Bootstrap samples
→ Random feature subsets
→ Multiple decision trees
→ Tree predictions
→ Majority voting
→ Final class

Random Forest reduces overfitting compared with a single decision tree because the final decision is based on many trees rather than one tree. It also improves robustness because different trees capture different parts of the feature space.

Boosting

Boosting is a sequential meta-algorithm. It builds classifiers one after another. Unlike bagging, where classifiers are trained independently, boosting trains each new classifier by giving more attention to the documents that were misclassified by earlier classifiers.

Initially, all training documents are given equal importance. After the first classifier is trained, the wrongly classified documents receive higher weights. The next classifier focuses more on these difficult documents. This process continues for several rounds. At the end, all weak classifiers are combined using weighted voting. Classifiers with lower error receive higher importance in the final decision.

The working principle of boosting is:

Training documents with equal weights → Weak classifier 1
→ Increase weights of misclassified documents
→ Weak classifier 2
→ Repeat weight adjustment
→ Weighted combination of classifiers
→ Final class

AdaBoost is a commonly used boosting algorithm. In text classification, boosting is useful when simple weak rules are available. For example, one weak rule may classify a ticket as Refund if the term *refund* is present, while another weak rule may classify a ticket as Delivery if the term *delay*

is present. Individually, these rules may not be perfect, but boosting combines them into a stronger classifier.

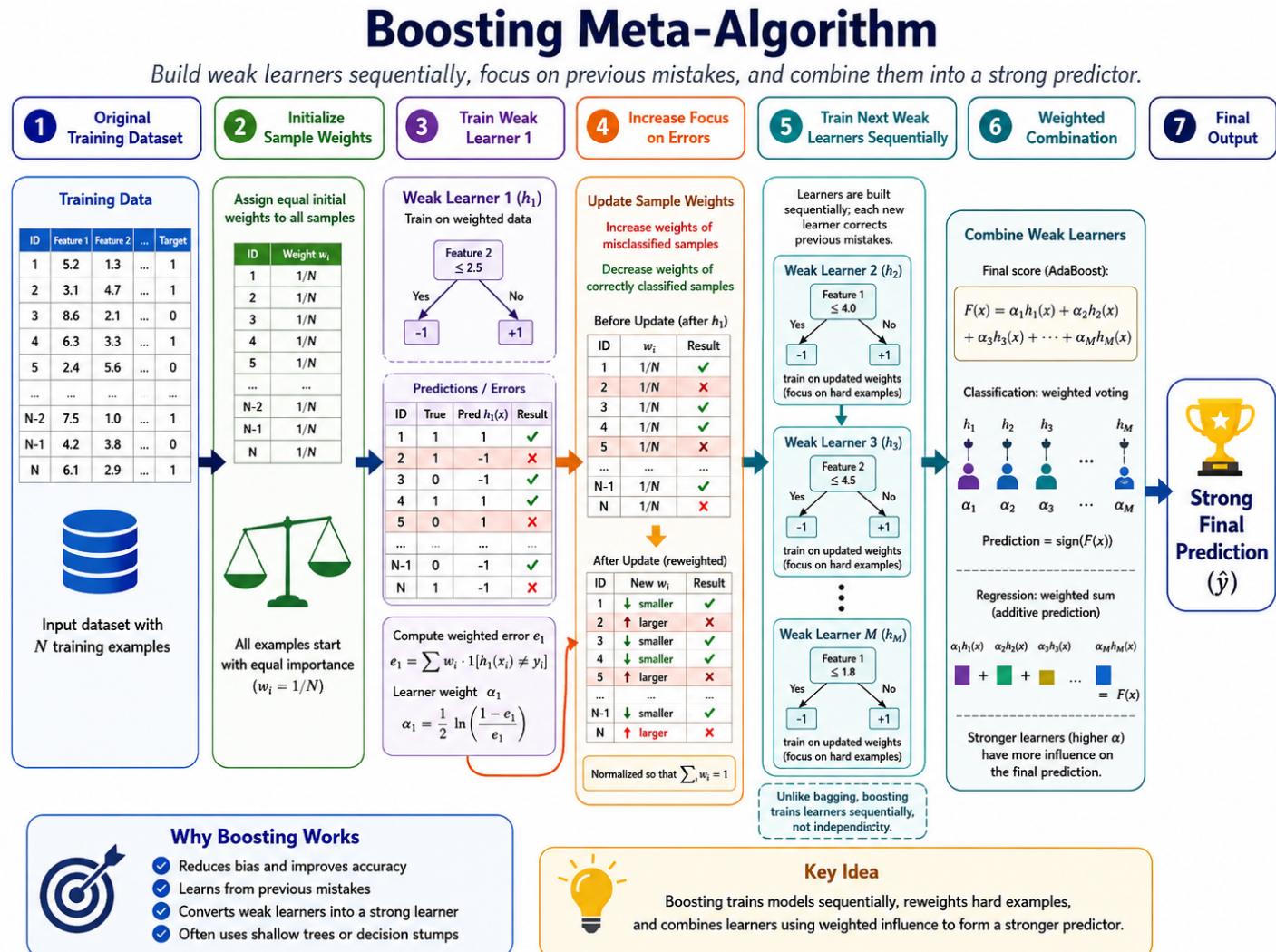


Figure 2: Working principle of boosting in text classification

Voting

Voting is a meta-algorithm that combines the predictions of different classifiers. Instead of training only one classifier, several different classifiers are trained on the same dataset. The final class is decided based on their combined predictions.

In *hard voting*, each classifier gives one class label, and the class receiving the majority of votes is selected. For example:

Naive Bayes → Refund

SVM → Delivery

Decision Tree → Refund

Since Refund receives two votes, the final class is:

Refund

In *soft voting*, classifiers produce class probabilities, and the probabilities are averaged. The class with the highest average probability is selected.

The working principle of voting is:

Document-term matrix → Different classifiers
 → Classifier predictions
 → Vote aggregation
 → Final class

Voting is useful when different classifiers capture different patterns in the same text dataset. A probabilistic classifier may capture word likelihoods, a linear classifier may capture weighted feature combinations, and a tree-based classifier may capture rule-like decisions.

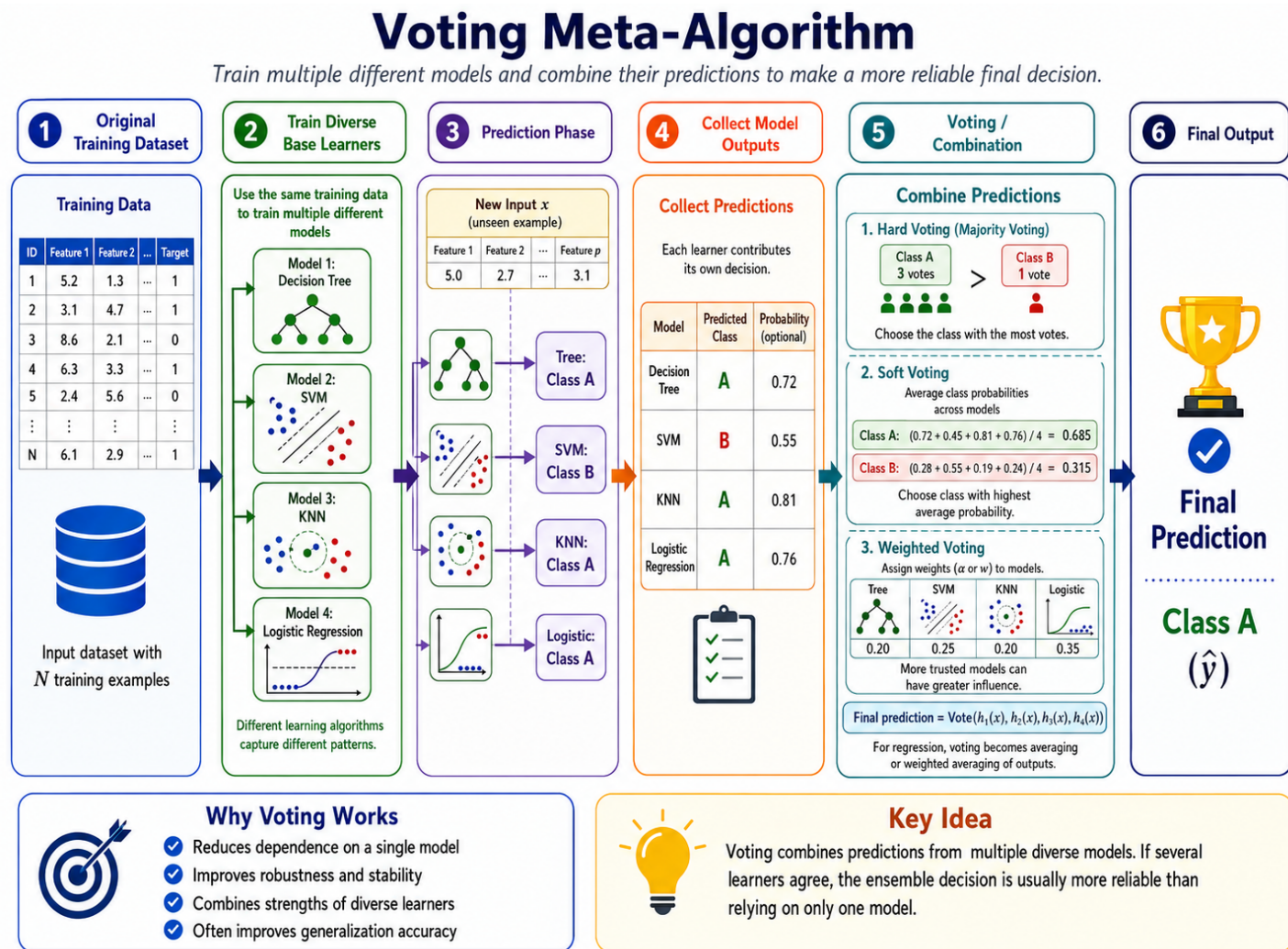


Figure 3: Working principle of voting-based text classification

Stacking

Stacking, also called *stacked generalization*, is a meta-learning approach. It trains multiple base classifiers first. The predictions or probability outputs of these base classifiers are then used as input features for another classifier called the *meta-classifier*.

The base classifiers operate on the original document-term matrix. The meta-classifier does not directly use the original words. Instead, it learns from the outputs of the base classifiers. For example, if Naive Bayes, SVM, and Random Forest produce predictions for a document, the meta-classifier learns how to combine those predictions effectively.

The working principle of stacking is:

Document-term matrix $\rightarrow$ Base classifiers
 $\rightarrow$ Base classifier outputs
 $\rightarrow$ Meta-classifier
 $\rightarrow$ Final class

Stacking is more flexible than simple voting because it does not treat all classifiers equally. The meta-classifier learns which base classifier should be trusted more under different conditions. However, stacking requires careful training to avoid overfitting.

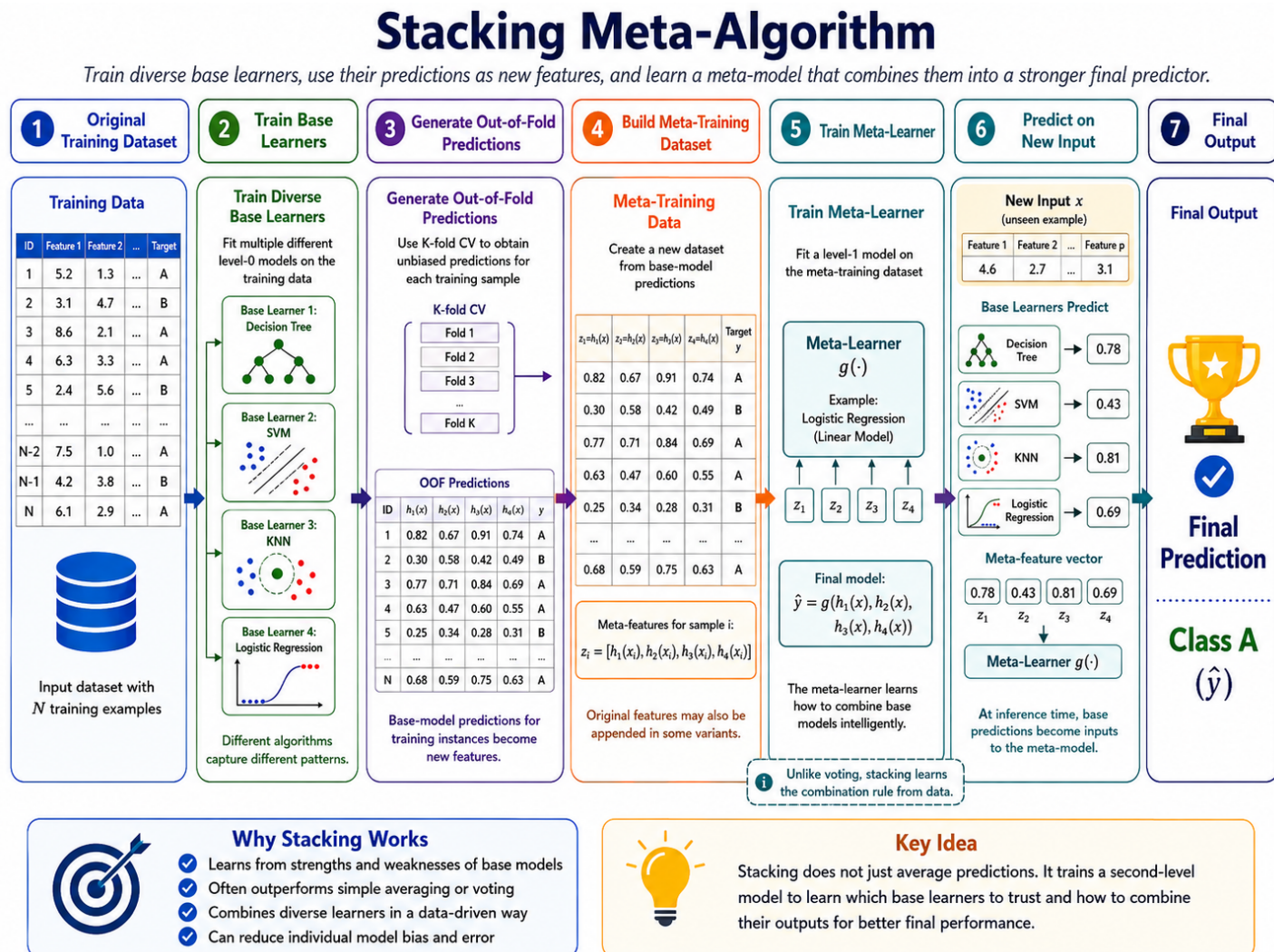


Figure 4: Working principle of stacking for text classification

Difference Among Meta-Algorithms

Meta-algorithm	Working principle	Final decision method
Bagging	Trains multiple classifiers on bootstrap samples of the dataset	Majority voting
Random Forest	Trains multiple decision trees using bootstrap samples and random feature subsets	Majority voting among trees
Boosting	Trains weak classifiers sequentially by focusing more on misclassified documents	Weighted voting
Voting	Combines predictions from different classifiers	Majority voting or probability averaging
Stacking	Uses outputs of base classifiers as inputs to a meta-classifier	Learned decision by meta-classifier

Table 23: Comparison of meta-algorithms for text classification

5.2 Random Forest Classifier

Random Forest is a bagging-based ensemble classifier. It builds multiple decision trees using different bootstrap samples and random feature subsets. In text classification, the features are terms obtained from the document-term matrix. Each tree gives a class prediction, and the final class is decided using majority voting.

Random Forest improves a single decision tree in two ways. First, each tree is trained using a different sample of the training data. Second, each tree considers only a random subset of terms while selecting a split. This makes the trees diverse and reduces overfitting.

Dataset

Consider a customer-support ticket classification problem. Each ticket belongs to one of two classes:

Refund, Delivery

Document	Raw Customer-Support Ticket	Class
D1	Refund is pending after payment.	Refund
D2	Refund account is pending.	Refund
D3	Refund payment issue reported.	Refund
D4	Delivery delay by courier.	Delivery
D5	Delivery tracking shows delay.	Delivery
D6	Courier delivery payment issue.	Delivery

Table 24: Customer-support ticket dataset for Random Forest

Preprocessing

After lowercase conversion, punctuation removal, stop-word removal, and simple normalization, the documents become:

Document	Preprocessed Ticket	Class
D1	refund pending payment	Refund
D2	refund account pending	Refund
D3	refund payment issue	Refund
D4	delivery delay courier	Delivery
D5	delivery tracking delay	Delivery
D6	courier delivery payment issue	Delivery

Table 25: Preprocessed customer-support tickets

Vocabulary and Vector Representation

The vocabulary is constructed from all preprocessed documents.

$$V = [\text{refund, pending, payment, account, issue, delivery, delay, courier, tracking}]$$

The vocabulary size is:

$$|V| = 9$$

The binary document-term matrix is:

Doc	refund	pending	payment	account	issue	delivery	delay	courier	tracking	Class
D1	1	1	1	0	0	0	0	0	0	Refund
D2	1	1	0	1	0	0	0	0	0	Refund
D3	1	0	1	0	1	0	0	0	0	Refund
D4	0	0	0	0	0	1	1	1	0	Delivery
D5	0	0	0	0	0	1	1	0	1	Delivery
D6	0	0	1	0	1	1	0	1	0	Delivery

Table 26: Binary document-term matrix

The query ticket to be classified is:

$$Q = \text{payment delivery delay}$$

Using the vocabulary order, its binary vector is:

$$Q = [0, 0, 1, 0, 0, 1, 1, 0, 0]$$

Gini Impurity

Random Forest usually uses decision trees as base learners. A decision tree selects a splitting term using an impurity measure such as Gini impurity.

$$Gini = 1 - p_{\text{Refund}}^2 - p_{\text{Delivery}}^2$$

At the root node:

$$\text{Refund} = 3, \quad \text{Delivery} = 3$$

$$p_{\text{Refund}} = \frac{3}{6} = 0.5$$

$$p_{\text{Delivery}} = \frac{3}{6} = 0.5$$

$$Gini_{\text{root}} = 1 - (0.5)^2 - (0.5)^2$$

$$Gini_{\text{root}} = 1 - 0.25 - 0.25 = 0.500$$

Tree 1 Construction

For Tree 1, assume that the random feature subset is:

[refund, delivery, delay]

The split using the term *refund* is examined.

If *refund* is present:

D1, D2, D3

All three documents belong to Refund.

Refund = 3, Delivery = 0

$$Gini(\text{refund} = 1) = 1 - \left(\frac{3}{3}\right)^2 - \left(\frac{0}{3}\right)^2$$

$$Gini(\text{refund} = 1) = 1 - 1 - 0 = 0$$

If *refund* is absent:

D4, D5, D6

All three documents belong to Delivery.

Refund = 0, Delivery = 3

$$Gini(\text{refund} = 0) = 1 - \left(\frac{0}{3}\right)^2 - \left(\frac{3}{3}\right)^2$$

$$Gini(\text{refund} = 0) = 1 - 0 - 1 = 0$$

The weighted Gini after splitting by *refund* is:

$$Gini_{\text{split}} = \frac{3}{6}(0) + \frac{3}{6}(0)$$

$$Gini_{\text{split}} = 0$$

Thus, Tree 1 selects *refund* as the splitting term.

$$h_1 : \begin{cases} \text{Refund,} & \text{if refund is present} \\ \text{Delivery,} & \text{if refund is absent} \end{cases}$$

For the query $Q = \text{payment delivery delay}$, the term *refund* is absent.

$$\text{Tree 1} \rightarrow \text{Delivery}$$

Tree 2 Construction

For Tree 2, assume that the random feature subset is:

$$[\text{payment, delay, courier}]$$

Detailed computation is shown for the term *delay*.

If *delay* is present:

$$D4, D5$$

Both belong to Delivery.

$$\text{Refund} = 0, \quad \text{Delivery} = 2$$

$$Gini(\text{delay} = 1) = 1 - \left(\frac{0}{2}\right)^2 - \left(\frac{2}{2}\right)^2$$

$$Gini(\text{delay} = 1) = 0$$

If *delay* is absent:

$$D1, D2, D3, D6$$

The class distribution is:

$$\text{Refund} = 3, \quad \text{Delivery} = 1$$

$$Gini(\text{delay} = 0) = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2$$

$$= 1 - \frac{9}{16} - \frac{1}{16}$$

$$= \frac{6}{16} = 0.375$$

The weighted Gini is:

$$Gini_{\text{delay}} = \frac{2}{6}(0) + \frac{4}{6}(0.375)$$

$$Gini_{\text{delay}} = 0.250$$

The remaining feature computations are shown compactly.

Feature	Present Branch	Absent Branch	Weighted Gini Computation	Weighted Gini
payment	D1, D3, D6: $R = 2, D = 1$, $Gini = 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 0.444$	D2, D4, D5: $R = 1, D = 2$, $Gini = 0.444$	$\frac{3}{6}(0.444) + \frac{3}{6}(0.444)$	0.444
delay	D4, D5: $R = 0, D = 2$, $Gini = 0$	D1, D2, D3, D6: $R = 3, D = 1$, $Gini = 0.375$	$\frac{2}{6}(0) + \frac{4}{6}(0.375)$	0.250
courier	D4, D6: $R = 0, D = 2$, $Gini = 0$	D1, D2, D3, D5: $R = 3, D = 1$, $Gini = 0.375$	$\frac{2}{6}(0) + \frac{4}{6}(0.375)$	0.250

Table 27: Gini computation for Tree 2

The minimum weighted Gini is:

$$0.250$$

There is a tie between *delay* and *courier*. The term *delay* is selected.

$$h_2 : \begin{cases} \text{Delivery,} & \text{if delay is present} \\ \text{Refund,} & \text{if delay is absent} \end{cases}$$

For the query Q , the term *delay* is present.

$$Tree\ 2 \rightarrow \text{Delivery}$$

Tree 3 Construction

For Tree 3, assume that the random feature subset is:

$$[\text{pending, courier, tracking}]$$

The possible splits are computed below.

Feature	Present Branch	Absent Branch	Weighted Gini Computation	Weighted Gini
pending	D1, D2: $R = 2, D = 0, Gini = 0$	D3, D4, D5, D6: $R = 1, D = 3, Gini = 1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2 = 0.375$	$\frac{2}{6}(0) + \frac{4}{6}(0.375)$	0.250
courier	D4, D6: $R = 0, D = 2, Gini = 0$	D1, D2, D3, D5: $R = 3, D = 1, Gini = 0.375$	$\frac{2}{6}(0) + \frac{4}{6}(0.375)$	0.250
tracking	D5: $R = 0, D = 1, Gini = 0$	D1, D2, D3, D4, D6: $R = 3, D = 2, Gini = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0.480$	$\frac{1}{6}(0) + \frac{5}{6}(0.480)$	0.400

Table 28: Gini computation for Tree 3

The minimum weighted Gini is:

$$0.250$$

There is a tie between *pending* and *courier*. The term *pending* is selected.

$$h_3 : \begin{cases} \text{Refund,} & \text{if pending is present} \\ \text{Delivery,} & \text{if pending is absent} \end{cases}$$

For the query Q , the term *pending* is absent.

$$Tree\ 3 \rightarrow \text{Delivery}$$

Random Forest Voting

The tree predictions are:

$$Tree\ 1 \rightarrow \text{Delivery}$$

$$Tree\ 2 \rightarrow \text{Delivery}$$

$$Tree\ 3 \rightarrow \text{Delivery}$$

The majority vote is:

$$\text{Delivery} = 3, \quad \text{Refund} = 0$$

Therefore, the Random Forest prediction is:

$$Q \rightarrow \text{Delivery}$$

5.3 AdaBoost Classifier

AdaBoost, or Adaptive Boosting, is a boosting algorithm that combines weak classifiers sequentially. Initially, all training documents have equal weights. After each round, the weights of wrongly classified documents are increased so that the next classifier gives more attention to difficult documents.

The same dataset, preprocessing, vocabulary, and binary document-term matrix used in the Random Forest subsection are used for AdaBoost.

The class labels are encoded as:

$$\text{Refund} = +1$$

$$\text{Delivery} = -1$$

The final classifier is a weighted combination of weak classifiers:

$$H(d) = \text{sign}(\alpha_1 h_1(d) + \alpha_2 h_2(d) + \cdots + \alpha_m h_m(d))$$

where:

$$h_i(d) = \text{prediction of weak classifier } i$$

$$\alpha_i = \text{weight of weak classifier } i$$

Weak Classifiers

The following weak classifiers are used:

$$h_1 : \begin{cases} +1, & \text{if payment is present} \\ -1, & \text{if payment is absent} \end{cases}$$

$$h_2 : \begin{cases} -1, & \text{if delay is present} \\ +1, & \text{if delay is absent} \end{cases}$$

$$h_3 : \begin{cases} +1, & \text{if pending is present} \\ -1, & \text{if pending is absent} \end{cases}$$

Initial Weights

There are six training documents. Initially, each document receives equal weight:

$$w_i = \frac{1}{6} = 0.167$$

Document	Class Label	Initial Weight
D1	+1	0.167
D2	+1	0.167
D3	+1	0.167
D4	-1	0.167
D5	-1	0.167
D6	-1	0.167

Table 29: Initial document weights

Round 1: Applying h_1

The first weak classifier is:

$$h_1 : \begin{cases} +1, & \text{if payment is present} \\ -1, & \text{if payment is absent} \end{cases}$$

Document	Actual Label	Payment Present	h_1 Prediction	Result
D1	+1	Yes	+1	Correct
D2	+1	No	-1	Wrong
D3	+1	Yes	+1	Correct
D4	-1	No	-1	Correct
D5	-1	No	-1	Correct
D6	-1	Yes	+1	Wrong

Table 30: Round 1 predictions using h_1

Wrongly classified documents are:

$$D2, \quad D6$$

The weighted error is:

$$Error(h_1) = w(D2) + w(D6)$$

$$Error(h_1) = 0.167 + 0.167 = 0.334$$

Using exact fraction:

$$Error(h_1) = \frac{2}{6} = 0.333$$

The classifier weight is:

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - Error(h_1)}{Error(h_1)} \right)$$

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - 0.333}{0.333} \right)$$

$$\alpha_1 = \frac{1}{2} \ln(2)$$

$$\alpha_1 = 0.347$$

Correctly classified documents are multiplied by:

$$e^{-\alpha_1} = e^{-0.347} = 0.707$$

Wrongly classified documents are multiplied by:

$$e^{\alpha_1} = e^{0.347} = 1.414$$

Unnormalized updated weights:

$$D1 = 0.167 \cdot 0.707 = 0.118$$

$$D2 = 0.167 \cdot 1.414 = 0.236$$

$$D3 = 0.167 \cdot 0.707 = 0.118$$

$$D4 = 0.167 \cdot 0.707 = 0.118$$

$$D5 = 0.167 \cdot 0.707 = 0.118$$

$$D6 = 0.167 \cdot 1.414 = 0.236$$

Total unnormalized weight:

$$0.118 + 0.236 + 0.118 + 0.118 + 0.118 + 0.236 = 0.944$$

The normalized weights are:

Document	Unnormalized Weight	Normalized Weight
D1	0.118	$\frac{0.118}{0.944} = 0.125$
D2	0.236	$\frac{0.236}{0.944} = 0.250$
D3	0.118	$\frac{0.118}{0.944} = 0.125$
D4	0.118	$\frac{0.118}{0.944} = 0.125$
D5	0.118	$\frac{0.118}{0.944} = 0.125$
D6	0.236	$\frac{0.236}{0.944} = 0.250$

Table 31: Weight update after Round 1

The weights of D2 and D6 increase because these documents were misclassified.

Round 2: Applying h_2

The second weak classifier is:

$$h_2 : \begin{cases} -1, & \text{if delay is present} \\ +1, & \text{if delay is absent} \end{cases}$$

Document	Actual Label	Delay Present	h_2 Prediction	Current Weight	Result
D1	+1	No	+1	0.125	Correct
D2	+1	No	+1	0.250	Correct
D3	+1	No	+1	0.125	Correct
D4	-1	Yes	-1	0.125	Correct
D5	-1	Yes	-1	0.125	Correct
D6	-1	No	+1	0.250	Wrong

Table 32: Round 2 predictions using h_2

Wrongly classified document:

D_6

The weighted error is:

$$Error(h_2) = w(D_6) = 0.250$$

The classifier weight is:

$$\alpha_2 = \frac{1}{2} \ln \left(\frac{1 - 0.250}{0.250} \right)$$

$$\alpha_2 = \frac{1}{2} \ln(3)$$

$$\alpha_2 = 0.549$$

Correctly classified documents are multiplied by:

$$e^{-0.549} = 0.577$$

Wrongly classified documents are multiplied by:

$$e^{0.549} = 1.732$$

Unnormalized updated weights:

$$D1 = 0.125 \cdot 0.577 = 0.072$$

$$D2 = 0.250 \cdot 0.577 = 0.144$$

$$D3 = 0.125 \cdot 0.577 = 0.072$$

$$D4 = 0.125 \cdot 0.577 = 0.072$$

$$D5 = 0.125 \cdot 0.577 = 0.072$$

$$D6 = 0.250 \cdot 1.732 = 0.433$$

Total unnormalized weight:

$$0.072 + 0.144 + 0.072 + 0.072 + 0.072 + 0.433 = 0.865$$

The normalized weights are:

Document	Unnormalized Weight	Normalized Weight
D1	0.072	$\frac{0.072}{0.865} = 0.083$
D2	0.144	$\frac{0.144}{0.865} = 0.167$
D3	0.072	$\frac{0.072}{0.865} = 0.083$
D4	0.072	$\frac{0.072}{0.865} = 0.083$
D5	0.072	$\frac{0.072}{0.865} = 0.083$
D6	0.433	$\frac{0.433}{0.865} = 0.500$

Table 33: Weight update after Round 2

The weight of D6 becomes high because it remains difficult to classify.

Round 3: Applying h_3

The third weak classifier is:

$$h_3 : \begin{cases} +1, & \text{if pending is present} \\ -1, & \text{if pending is absent} \end{cases}$$

Document	Actual Label	Pending Present	h_3 Prediction	Current Weight	Result
D1	+1	Yes	+1	0.083	Correct
D2	+1	Yes	+1	0.167	Correct
D3	+1	No	-1	0.083	Wrong
D4	-1	No	-1	0.083	Correct
D5	-1	No	-1	0.083	Correct
D6	-1	No	-1	0.500	Correct

Table 34: Round 3 predictions using h_3

Wrongly classified document:

D_3

The weighted error is:

$$Error(h_3) = w(D_3) = 0.083$$

The classifier weight is:

$$\alpha_3 = \frac{1}{2} \ln \left(\frac{1 - 0.083}{0.083} \right)$$

$$\alpha_3 = \frac{1}{2} \ln(11)$$

$$\alpha_3 = 1.199$$

Final AdaBoost Classifier

The final classifier is:

$$H(d) = \text{sign}(0.347h_1(d) + 0.549h_2(d) + 1.199h_3(d))$$

A positive score indicates Refund. A negative score indicates Delivery.

For the query:

Q = payment delivery delay

The weak classifier outputs are:

$$h_1(Q) = +1$$

because *payment* is present.

$$h_2(Q) = -1$$

because *delay* is present.

$$h_3(Q) = -1$$

because *pending* is absent.

Final score:

$$\text{Score}(Q) = 0.347(+1) + 0.549(-1) + 1.199(-1)$$

$$\text{Score}(Q) = 0.347 - 0.549 - 1.199$$

$$\text{Score}(Q) = -1.401$$

Since the score is negative:

$$Q \rightarrow \text{Delivery}$$

Interpretation

Random Forest and AdaBoost are both meta-algorithms, but they improve classification differently. Random Forest builds several decision trees using random feature subsets and combines their predictions using majority voting. AdaBoost builds weak classifiers sequentially, increases the weight of wrongly classified documents, and combines the weak classifiers using weighted voting.

In the worked example, the mixed query *payment delivery delay* contains one refund-related word and two delivery-related words. Random Forest predicts Delivery through majority voting. AdaBoost also predicts Delivery because the weighted influence of the delay-based and pending-based weak classifiers is stronger than the payment-based weak classifier.

6 Content-Based Spam Email Classification Using Machine-Learning Algorithms

6.1 Core Principle

Content-based spam email classification is a supervised text classification task in which an email is classified as *Spam* or *Not Spam* using the words present in the email content. The classification

decision is based on the subject line, message body, and textual patterns. It does not depend on sender address, IP address, attachment type, or network-level information.

Spam emails usually contain terms such as *claim*, *free*, *reward*, *urgent*, *offer*, and *verify*. Legitimate emails usually contain work-related or transaction-related terms such as *project*, *meeting*, *invoice*, *payment*, *report*, and *review*. A machine-learning classifier learns the relationship between such terms and class labels from labelled email examples.

The general workflow is:

Raw email → Preprocessing → Vocabulary construction
→ Document-term representation → Model training → Spam or Not Spam decision

In this section, two machine-learning algorithms are discussed with computation:

Logistic Regression

Support Vector Machine

6.2 Dataset and Vector Representation

Consider the following email dataset. Each email is labelled as either *Spam* or *Not Spam*.

Email	Raw Email Text	Class
E1	Claim your free reward now.	Spam
E2	Urgent free offer and reward.	Spam
E3	Verify account and claim urgent reward.	Spam
E4	Project meeting schedule and report.	Not Spam
E5	Invoice payment review report.	Not Spam
E6	Project review meeting payment.	Not Spam

Table 35: Email dataset for content-based spam classification

Preprocessing

After lowercase conversion, punctuation removal, and stop-word removal, the emails become:

Email	Preprocessed Email Text	Class
E1	claim free reward now	Spam
E2	urgent free offer reward	Spam
E3	verify account claim urgent reward	Spam
E4	project meeting schedule report	Not Spam
E5	invoice payment review report	Not Spam
E6	project review meeting payment	Not Spam

Table 36: Preprocessed email dataset

Vocabulary Construction

The vocabulary is constructed from all preprocessed emails.

$V = [\text{claim, free, reward, now, urgent, offer,}$
 $\text{verify, account, project, meeting, schedule,}$
 $\text{report, invoice, payment, review}]$

The vocabulary size is:

$$|V| = 15$$

All vocabulary terms are used for representation. No shortened or manually selected term list is used.

Binary Email-Term Matrix

The value 1 indicates that the term appears in the email, and 0 indicates that the term does not appear.

Email	claim	free	reward	now	urgent	offer	verify	account	project	meeting	schedule	report	invoice	payment	review	Class
E1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	Spam
E2	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	Spam
E3	1	0	1	0	1	0	1	1	0	0	0	0	0	0	0	Spam
E4	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	Not Spam
E5	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	Not Spam
E6	0	0	0	0	0	0	0	0	1	1	0	0	0	1	1	Not Spam

Table 37: Binary email-term matrix

The following query emails are used for classification:

Q_1 = claim free offer

Q_2 = project invoice review

Q_3 = urgent payment review

6.3 Logistic Regression Classifier

Logistic Regression is a supervised machine-learning algorithm used for binary classification. In spam classification, it computes a linear score from the email-term vector and converts that score into a probability using the sigmoid function.

The class encoding is:

Spam = 1

Not Spam = 0

For an email vector x , the linear score is:

$$z = b + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

where:

b = bias term

w_i = weight of term i

x_i = value of term i

The sigmoid function converts the score into a probability:

$$P(\text{Spam} \mid x) = \frac{1}{1 + e^{-z}}$$

The decision rule is:

$$P(\text{Spam} \mid x) \geq 0.5 \Rightarrow \text{Spam}$$

$$P(\text{Spam} \mid x) < 0.5 \Rightarrow \text{Not Spam}$$

Trained Logistic Regression Parameters

After training on the email dataset, assume that the Logistic Regression model learns the following weights. Positive weights support the Spam class, and negative weights support the Not Spam class.

$$b = 0$$

Term	Weight	Term	Weight
claim	1.4	account	0.7
free	1.5	project	-1.3
reward	1.2	meeting	-1.2
now	0.8	schedule	-0.9
urgent	1.3	report	-1.0
offer	1.1	invoice	-1.1
verify	1.0	payment	-1.0
review	-1.2		

Table 38: Illustrative trained weights for Logistic Regression

Detailed Computation for Q_1

The query email is:

$$Q_1 = \text{claim free offer}$$

The active terms are:

claim, free, offer

The corresponding weights are:

$$w_{\text{claim}} = 1.4, \quad w_{\text{free}} = 1.5, \quad w_{\text{offer}} = 1.1$$

The linear score is:

$$z = 0 + 1.4 + 1.5 + 1.1$$

$$z = 4.0$$

The Spam probability is:

$$P(\text{Spam} \mid Q_1) = \frac{1}{1 + e^{-4.0}}$$

Since:

$$e^{-4.0} = 0.018$$

$$P(\text{Spam} \mid Q_1) = \frac{1}{1 + 0.018}$$

$$P(\text{Spam} \mid Q_1) = 0.982$$

Since:

$$0.982 \geq 0.5$$

the email is classified as:

$$Q_1 \rightarrow \text{Spam}$$

Detailed Computation for Q_2

The query email is:

$$Q_2 = \text{project invoice review}$$

The active terms are:

$$\text{project, invoice, review}$$

The corresponding weights are:

$$w_{\text{project}} = -1.3, \quad w_{\text{invoice}} = -1.1, \quad w_{\text{review}} = -1.2$$

The linear score is:

$$z = 0 + (-1.3) + (-1.1) + (-1.2)$$

$$z = -3.6$$

The Spam probability is:

$$P(\text{Spam} \mid Q_2) = \frac{1}{1 + e^{-(-3.6)}}$$

$$P(\text{Spam} \mid Q_2) = \frac{1}{1 + e^{3.6}}$$

Since:

$$e^{3.6} = 36.598$$

$$P(\text{Spam} \mid Q_2) = \frac{1}{1 + 36.598}$$

$$P(\text{Spam} \mid Q_2) = 0.027$$

Since:

$$0.027 < 0.5$$

the email is classified as:

$$Q_2 \rightarrow \text{Not Spam}$$

Compact Logistic Regression Classification Table

The same computation is applied to all query emails.

Query	Active Terms	Linear Score Computation	z	$P(\text{Spam} \mid x)$	Class
Q_1	claim, free, offer	$0 + 1.4 + 1.5 + 1.1$	4.0	$\frac{1}{1+e^{-4.0}} = 0.982$	Spam
Q_2	project, invoice, review	$0 - 1.3 - 1.1 - 1.2$	-3.6	$\frac{1}{1+e^{3.6}} = 0.027$	Not Spam
Q_3	urgent, payment, review	$0 + 1.3 - 1.0 - 1.2$	-0.9	$\frac{1}{1+e^{0.9}} = 0.289$	Not Spam

Table 39: Logistic Regression classification of query emails

The mixed query Q_3 contains the Spam-related term *urgent*, but the Not-Spam-related terms *payment* and *review* give stronger negative contribution. Therefore, the final probability is below 0.5 and the email is classified as Not Spam.

6.4 Support Vector Machine Classifier

Support Vector Machine, commonly called *SVM*, is a supervised machine-learning algorithm that finds a separating boundary between classes. In text classification, a linear SVM is commonly used because document-term vectors are high-dimensional and sparse.

The class encoding is:

$$\text{Spam} = +1$$

$$\text{Not Spam} = -1$$

A linear SVM computes the decision score:

$$f(x) = b + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

The decision rule is:

$$f(x) > 0 \Rightarrow \text{Spam}$$

$$f(x) < 0 \Rightarrow \text{Not Spam}$$

Unlike Logistic Regression, SVM does not mainly focus on probability estimation. It focuses on finding a separating hyperplane with maximum margin.

Trained Linear SVM Parameters

Assume that the trained linear SVM learns the following weights:

$$b = 0$$

Term	SVM Weight	Term	SVM Weight
claim	2.0	account	1.0
free	2.1	project	-1.8
reward	1.8	meeting	-1.7
now	1.0	schedule	-1.2
urgent	1.9	report	-1.5
offer	1.6	invoice	-1.6
verify	1.4	payment	-1.3
review	-1.6		

Table 40: Illustrative trained weights for linear SVM

Positive weights push the decision toward Spam, while negative weights push the decision toward Not Spam.

Detailed Computation for Q_1

The query email is:

$$Q_1 = \text{claim free offer}$$

The active terms are:

claim, free, offer

The decision score is:

$$f(Q_1) = 0 + 2.0 + 2.1 + 1.6$$

$$f(Q_1) = 5.7$$

Since:

$$5.7 > 0$$

the email is classified as:

$$Q_1 \rightarrow \text{Spam}$$

Detailed Computation for Q_2

The query email is:

$$Q_2 = \text{project invoice review}$$

The active terms are:

project, invoice, review

The decision score is:

$$f(Q_2) = 0 + (-1.8) + (-1.6) + (-1.6)$$

$$f(Q_2) = -5.0$$

Since:

$$-5.0 < 0$$

the email is classified as:

$$Q_2 \rightarrow \text{Not Spam}$$

Compact SVM Classification Table

The same decision function is applied to all query emails.

Query	Active Terms	SVM Score Computation	$f(x)$	Class
Q_1	claim, free, offer	$0 + 2.0 + 2.1 + 1.6$	5.7	Spam
Q_2	project, invoice, review	$0 - 1.8 - 1.6 - 1.6$	-5.0	Not Spam
Q_3	urgent, payment, review	$0 + 1.9 - 1.3 - 1.6$	-1.0	Not Spam

Table 41: Linear SVM classification of query emails

The mixed email Q_3 contains *urgent*, which supports Spam. However, the terms *payment* and *review* provide stronger evidence toward Not Spam. Hence, the final SVM score is negative.

Interpretation

Content-based spam email classification depends on the textual evidence present in the email. The email content is preprocessed, converted into a document-term representation, and then classified using a trained machine-learning model.

Logistic Regression computes a weighted linear score and converts it into a Spam probability using the sigmoid function. Linear SVM computes a decision score and assigns the email to the side of the separating boundary on which it falls.

In the worked example, terms such as *claim*, *free*, *reward*, *urgent*, and *offer* provide evidence for Spam. Terms such as *project*, *meeting*, *invoice*, *payment*, *report*, and *review* provide evidence for

Not Spam. Both Logistic Regression and SVM classify the queries according to the strength of these learned term weights.